

Enterprise Agent Hosting Platform

A Reference Architecture

Own the control plane. Rent the compute. Decouple through protocols.

An architecture for hosting AI agents inside an enterprise without handing control to any single vendor.

THE CORE PROPOSITION

A cloud vendor can host the machines. It cannot answer, on the enterprise's behalf, the questions that matter: *who may run, what may they do, what did they do, what did it cost, and what happened when something went wrong*. The job of an enterprise agent platform is to put uncertain, model-driven behavior inside a boundary that is deterministic, verifiable, and portable.

Own the control plane

Identity, authorization, the ledger, task state and the interface contracts stay with the enterprise. They *are* control.

Rent the compute

Runtimes, sandboxes, object storage and dashboards are rented by usage. Any vendor that meets the contract will do.

Decouple through protocols

The platform binds to a protocol, not to a framework. Any agent framework joins through an adapter; the contract never moves.

Author	Yuting	Version	v1.0
Date	September 2026	Status	Reference architecture and design proposal

This document describes a target architecture, its design constraints and a phased delivery boundary. It does not claim an existing production capability, measured performance, or any commercial service level.

Abstract

AI agents inside enterprises are moving from scattered experiments to managed hosting. What decides whether an agent can enter production is rarely the model itself. It is the set of questions around the model: whose identity does it act under, which systems may it touch, where do its credentials live, how far did a task get, what did it cost, how is an incident reconstructed, and what does it take to leave a vendor. When those capabilities are built on top of a single cloud runtime, the enterprise gains speed and loses control over its identities, policies, secrets and history.

This paper proposes a general reference architecture for an enterprise agent hosting platform. It is a design proposal, not an implementation report. Four decisions shape it. The enterprise keeps its own control plane—identity, authorization, the ledger, task state and the interface contracts—while renting machines, sandboxes, object storage and dashboards by usage. The platform binds to a protocol rather than to any agent framework. Every action an agent takes against the outside world—calling a model, calling a tool, executing code—passes through a governed exit that the platform owns. And every fact about a run is written to a ledger the enterprise holds.

On top of the managed runtimes, workload identity, credential vaults, gateways, sandboxes and durable execution that the industry already provides, the paper adds a set of enterprise-side mechanisms: a four-axis capability manifest that describes an agent along independent dimensions; a minimal container contract in which the platform, not the container, assigns event sequence numbers; a permission-intersection rule that covers user delegation, scheduled principals and structural denials; a three-stage kill switch that still works when the cloud vendor is unreachable; “effectively-once” side-effect semantics built from checkpoints, an idempotency ledger and archived results; and a hash-chained enterprise ledger that supports compliant deletion.

Keywords: enterprise agents; agent hosting platform; control plane; workload identity; tool governance; unified ledger; sandbox; cloud portability

How to read this document

The document is long, and different readers need different parts of it. Readers new to agent platforms should start with the glossary in Appendix A, then read Sections 1, 2 and 3, and then jump to Section 9, which walks a single request through every mechanism in the paper.

Reader	Suggested route
Executives and decision makers	Part I (the case and the object), Section 6 on the overall architecture, Section 20 on delivery, Section 23
Platform architects	The whole document, with emphasis on the ten invariants, the three interface layers, runtime semantics, identity and authorization, and the architecture decisions
Implementation teams	Parts II and III plus Appendices B and C: interfaces, runtime, identity, ledger and acceptance
Security and audit reviewers	Section 13 (threat model), Sections 14–16, Section 22 (residual risks)
First-time readers	Appendix A → Sections 1–3 → Section 9

Contents

Part I: The Case and the Object	1
1 Executive Summary	1
2 Problem Definition: Not a Shortage of Containers	2
3 What the Platform Hosts: Agents and Their Four Kinds	5
4 Design Method: Principles, Invariants and Boundaries	8
5 Related Work and Where This Paper Stands	11
Part II: Architecture and Runtime Contract	13
6 Overall Architecture	13
7 Capability Manifest and Admission	15
8 Three Interface Layers	19
9 One Request, End to End	23
10 Runtime Semantics	24
11 Reliability and Safety Controls	27
12 How Each Kind Fulfills the Contract	29
Part III: Security, Governance and Runtime Control	33
13 Threat Model and Trust Boundary	33
14 Identity and Authorization	33
15 Shared Services: The Governed Exits	36
16 The Unified Ledger, Metering and Observability	39
17 The Authoritative Store	41
18 The Execution Plane and the Vendor Boundary	42
Part IV: Productization, Delivery and Evolution	44
19 Enterprise Integration and Generalization	44
20 Phase 1 Delivery, Roadmap and Acceptance	46
21 Key Architecture Decisions and Trade-offs	46
22 Residual Risks and Companion Specifications	47
23 Conclusion	49
A Glossary	50
B L2 Container Contract Quick Reference	51
C Ten-Invariant Acceptance Checklist	52
D Revision History	53

List of Figures

1	The four kinds of agent	6
2	Selecting a kind	8
3	The six-layer boundary	13
4	Complete architecture	16
5	The governance layer	17
6	The four axes	18
7	The three interface layers	20
8	The entry layer	20
9	Record first, then distribute	22
10	Nine steps of one run	23
11	Runtime services	24
12	The five action channels	26
13	The run state machine	26
14	The three-stage kill switch	28
15	The three layers of a code agent	31
16	Two token classes	36
17	Credential flow in a tool call	37
18	The shared-services layer	37
19	Call path for environment tools	38
20	Three books of metering	40
21	The authoritative store	41
22	Execution plane and cloud resources	42
23	Enterprise systems and their integration points	44
24	Roadmap	46

List of Tables

1	Division of responsibility between a cloud runtime and the enterprise platform	3
2	What each stakeholder gets from the platform	4
3	Goals and non-goals	4
4	The four kinds compared	7
5	Eight design principles	9
6	Ten system invariants and how each is verified	10
7	Four ways to build	11
8	Capabilities the industry already provides, and what this paper does not claim	12
9	The five partitions of the control plane and their criteria	14
10	Module build matrix	17
11	The four axes of the capability manifest	18
12	Resource groups of the L1 unified API	21
13	Composition of the L2 contract	22
14	Persistence levels for the two classes of event	22
15	Session, thread and run	25
16	The five action channels and their governance	25
17	Authoritative ownership of the four kinds of state and what recovery means	27
18	Two kinds of idempotency	28
19	Interrupt and resume in three layers	29
20	How each kind fulfills the contract	29
21	Threat model	34
22	Four kinds of principal	34
23	The two token classes	35
24	Authoritative sources of each class of fact	39
25	The two product lines of the execution plane	43
26	Three verdicts on vendor components	43
27	How enterprise systems are integrated	45
28	Kernel, adapter packs and policy packs	45
29	Scope of the three phases	47
30	The six acceptance loops of Phase 1	47
31	Key architecture decisions	48
32	Residual risks	48

PART I

The Case and the Object

Why the platform should exist, what exactly it hosts, and how the four kinds of agent are told apart.

1 Executive Summary

An enterprise agent platform has to answer five questions: **who may run, what may they do, what did they do, what did it cost, and how is an incident reconstructed.** The five questions map onto four properties of the platform: controllable, meterable, observable, auditable. Everything in this paper exists so that those four properties hold inside the enterprise's own boundary.

A cloud vendor's agent runtime delivers requests into containers, keeps instances alive, scales them, and bills by "busy" versus "idle" time. It does these things well. What it does not know is who the agent in that container is, on whose behalf it acts, which systems it may touch, whose budget the run belongs to, or what happened inside. The vendor manages containers, not agents. Buying a vendor's complete agent product line therefore does not close the enterprise's governance gap by itself; it may instead lock identity, policy, secrets and history into that vendor.

The proposal rests on six judgments:

1. **The enterprise owns the control plane.** Registration, identity, authorization, task state, the ledger and the cloud adapter are long-term control points and cannot be outsourced.
2. **The execution plane is rented by usage.** Machines, sandboxes, object storage and read-only dashboards hold no decision rights; any vendor that meets the contract can supply them.
3. **The platform binds to a protocol, not to a framework.** A new agent framework joins through an adapter; the contract between platform and container does not change.
4. **Capability is declared and then proven.** An agent's capability manifest is an application; it takes effect only after a conformance test.
5. **Every side effect passes through a governed exit.** Calling a model, calling a tool, executing code—there is no path around the platform.
6. **Every fact goes into one ledger.** Administrator actions and agent actions share one identity chain, one event sequence and one set of audit rules.

In one sentence: an enterprise-owned control plane decides an agent's identity, version, capability and accountability; a minimal protocol connects replaceable cloud execution resources; governed exits constrain every external action; and one ledger holds every authoritative fact.

What the enterprise gets are four things that are hard to obtain together by any other means. *Fast onboarding*: an existing agent joins with a handful of changed lines, and a question-answering agent goes live from a prompt. *Affordable operation*: machines are rented by usage, waiting costs nothing, and every run has a hard ceiling. *Real control*: every external action passes an exit that can record and refuse it, and credentials never enter the agent. *Accountability*: any run can be replayed back to a specific person, a specific authorization and a specific cost, and none of that history is lost when the cloud vendor changes.

Phase 1 delivers only two kinds of agent (Q&A and code), a single cloud vendor, scheduled triggers and a basic console. A few things must nevertheless be right on day one, because fixing them later means rewriting every agent already in production: the event format of the container contract, the ownership of authoritative state, the single exits for models and tools, the hard ceiling on every run, the two kinds of idempotency, the separation of tokens by interface, the verification of user identity, a kill switch that works when the vendor is unreachable, and the ledger's hash chain. Workflow agents, interrupt and resume, human approval, long-term memory, multi-cloud and autonomous agents follow in Phases 2 and 3, once the contract is stable.

The document is organized as follows. Part I answers why the platform exists, what it hosts and which principles it follows, and relates the proposal to existing industry work. Part II is the architecture and runtime contract: the overall structure, capability manifests and admission, the three interface layers, one request end to end, runtime semantics, reliability and safety controls, and how each of the four agent kinds fulfills the contract. Part III covers security and governance: the threat model, identity and authorization, the governed exits, the unified ledger, the authoritative store, and the boundary with cloud vendors. Part IV discusses productization and delivery: integration with enterprise systems and generalization, phased delivery and acceptance, key decisions and residual risks. The appendices provide a glossary, a contract quick reference and an acceptance checklist.

2

Problem Definition: Not a Shortage of Containers

2.1 How agents differ from ordinary services

An ordinary API service has a fixed path. A request arrives, follows a predetermined branch of code, calls predetermined downstream systems, and consumes roughly predictable resources. An agent does not. It decides its next step from context: which model to use, which tool to call, whether to run a piece of code, whether to stop and ask a person. The same agent can take entirely different paths on different inputs.

That creates a new kind of risk. When an ordinary service fails, it usually gives a wrong answer. When an agent fails, it may have *done* the wrong thing, under a real identity, against a real system—placed an order, sent an email, changed a record, executed a script. Governing agents therefore shifts from “what did it say” to “what could it do, what did it do, and who authorized it”.

2.2 What happens when every team builds its own

If each team keeps its own secrets, chooses its own framework and deploys to its own cloud account, four kinds of fracture appear quickly:

- **Identity fracture.** Machine identities, developer identities and end-user identities blur together; afterwards nobody can prove who acted on whose behalf.
- **Governance fracture.** Release checks, tool authorizations, budgets and isolation policies are scattered across team code and vendor consoles, with no single point of enforcement.
- **Evidence fracture.** Models, tools, runtimes and file stores each keep their own logs in their own formats with their own clocks; a complete chain of responsibility cannot be reconstructed.
- **Vendor fracture.** Critical state and proprietary APIs settle into one cloud; switching vendors turns out to mean reimplementing the whole platform.

None of these fractures is fixed by buying a better runtime. They occur above the runtime, inside the enterprise's own boundary.

2.3 Why now

Three developments have turned this from a question that could wait into one that must be answered now.

First, agents are moving from experiments into production. While an agent is an internal chat assistant on trial, the cost of an error is an unsatisfactory answer. Once it touches customer data, internal systems and real transactions, the cost of an error is a real side effect. The need for governance grows with the agent's reach, not with the model's ability.

Second, cloud vendors have begun to offer agent infrastructure as complete product lines: runtime, sandbox, identity, gateway, memory and observability, packaged together. Each piece is useful on its own. But an enterprise that places its identities, policies, secrets and history into one vendor's product line will discover, when it wants to switch a few years later, that what needs rewriting is not one module but the whole platform. The window for deciding what to own and what to rent is now.

Third, audit and compliance requirements are becoming concrete. Regulators, internal audit and risk functions are asking the same question: who authorized this automated action, on what basis, and can it be reproduced? That question cannot be answered by stitching vendor logs together after the fact. It can only be answered by a chain of responsibility designed in from the start.

2.4 What the cloud runtime does and what the enterprise still must do

Table 1 places the two side by side. The left column is what a cloud runtime typically provides; the right column is what the enterprise platform remains responsible for.

Table 1 Division of responsibility between a cloud runtime and the enterprise platform

Capability	What the cloud runtime provides	What the enterprise platform still owns
Compute lifecycle	Deployment, scaling, liveness probes, busy/idle billing	Which agent, which version, and why it is running
Identity	Instance or cloud-account identity	User, service, scheduler and agent identities, and the "on whose behalf" chain
Authorization	Cloud resource access control	Which models and tools, which data, what risk tier, which structural denials
Credentials	A secrets product or environment variables	Credentials invisible to the agent, injected per call, every retrieval recorded
State	Ephemeral container or session state	Conversation history, run progress, checkpoints, artifacts, and the authoritative mapping among them
Observability	Machine logs and resource metrics	Business events, policy decisions, cost attribution, replay of responsibility
Migration	The vendor's own export features	A stable contract, enterprise-held state, adapters, and rehearsed exit

The cloud vendor manages containers; the enterprise platform manages agents. A container being alive does not mean the agent is allowed to act. A vendor bill being accurate does not mean the cost can be attributed to a team.

2.5 What the platform delivers

The opposite of the four fractures is the platform's value. Table 2 lists it by beneficiary: what each party gets, and which mechanisms in this paper provide it.

Table 2 What each stakeholder gets from the platform

Stakeholder	What they get	Through which mechanisms
Business teams	A Q&A agent goes live from a prompt; no machines, secrets or permissions to manage	Shared harness pool, capability manifest and admission, unified API
Development teams	An existing agent is hosted after a few changes, in any framework; releases are versioned and reversible	Protocol shell and framework adapters, cloud build, registry and release
Security and audit	Every external action passes a recordable, refusable exit; credentials never enter the agent; any run replays back to a person	Five action channels, credential vault, permission intersection, unified ledger with hash chain
Finance and cost control	Cost is attributed to tenant, agent and run; runaway runs have a ceiling; bills reconcile against the vendor	Three-book metering, circuit breaker, authoritative busy/idle judgment
Platform owners and management	Switching cloud vendors does not rewrite the platform; a rogue agent can be stopped without the vendor; the platform extends to other business lines	Container contract, cloud adapter, three-stage kill switch, kernel/policy-pack separation

2.6 Goals and non-goals

Stating what the platform will not do matters as much as stating what it will. Table 3 sets the boundary.

Table 3 Goals and non-goals

	In scope	Explicitly out of scope
Platform	A complete loop from development through release, operation, governance and accountability	A universal autonomous agent that covers every scenario
Frameworks	Tenants write agents in any framework and join through the protocol	Requiring tenants to standardize on one development framework
Compute	Runtimes and sandboxes rented by usage	Building and operating a stateful compute fleet in-house
Models and tools	Single exits with unified governance and metering	Training foundation models, or replacing the enterprise's existing tool systems
Multi-cloud	Contract first, single cloud first, migration provable	Maintaining several equivalent production stacks from day one
Long-running work	Each trigger creates one bounded run	Defining a resident daemon as a new kind of agent

3

What the Platform Hosts: Agents and Their Four Kinds

3.1 What counts as a hostable agent

In this paper an agent is not a model, a prompt or a chat page. It is a **versionable software unit that can be given an identity and that, within one run, chooses models or tools from its input and context and produces a result**. Something enters the platform's hosting boundary only when it satisfies all six of the following:

1. **A definition carrier.** It is defined by a prompt, a declarative workflow, source code, or a third-party process.
2. **A stable identity.** It has its own agent ID and does not borrow a developer's or a machine's account.
3. **An accountable owner.** A named human owner and a tenant, so that someone can be found when something goes wrong.
4. **Immutable versions.** It can be released, canaried and rolled back, and each version maps to the harness version it runs on.
5. **An execution boundary.** Each piece of work is a run with a start, a terminal state, a budget and the ability to be canceled.
6. **Operational evidence.** Its events, tool side effects, costs and artifacts can be written to the unified ledger.

The definition says nothing about how “intelligent” the agent is. The model generates; the agent decides and orchestrates; the platform bounds its identity, resources, side effects and accountability. What the platform governs is a software unit that can act on behalf of a principal—not a degree of intelligence.

3.2 How the four kinds are told apart: who decides the next step

A running agent is a loop: look at the current state, decide the next step, act, look again. The four kinds are separated by a single question: **who decides the control flow of that loop?** There are exactly four possible answers—the platform's fixed loop, a workflow declared in advance, the tenant's own code, or a third-party agent's own judgment. The split is independent of model strength, business value and risk level, which is why it is complete: there is no fifth “who decides”.

One term recurs throughout the paper. The **harness** is the general execution loop that the platform implements itself: the code that calls models and orchestrates tools. The harness belongs to the platform; tenants submit input, receive events or declare capabilities through a stable protocol and never modify the harness.

Reading Figure 1 from left to right, the platform's grip on the loop loosens and the autonomy of the agent's environment grows. That determines what the platform can use to govern each kind: for T0 the loop is platform code, so a violation is structurally impossible; for T1 the workflow is data and can be analyzed before release; for T2 the code is a program, constrainable only from outside through isolation and single exits; for T3 even the code is not in the platform's hands, leaving full-machine isolation and controlled exits as the only levers.

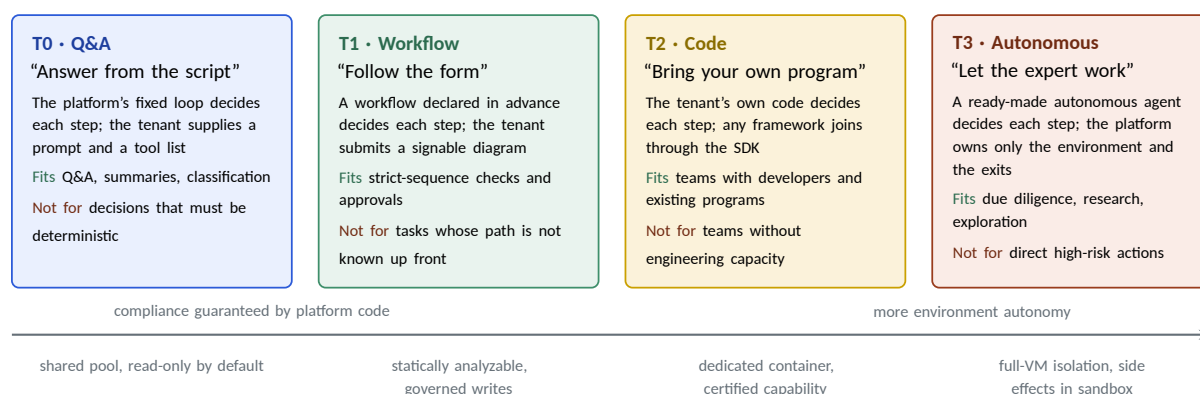
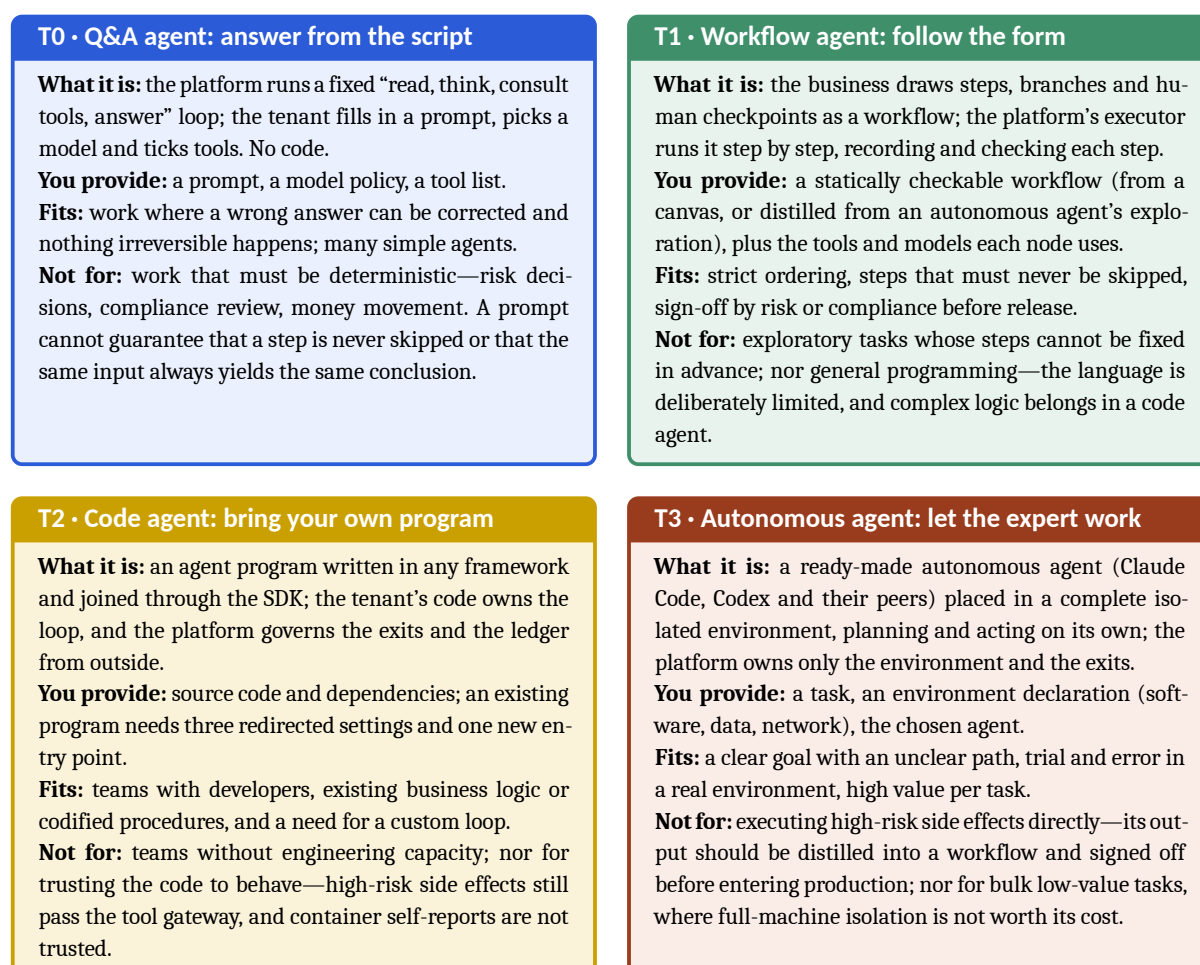


Figure 1 T0–T3 is a spectrum of “who decides each step”, not a ladder of capability or maturity

3.3 What each kind is and what it is for

A short way to remember the four: **Q&A agents answer from the script, workflow agents follow the form, code agents bring their own program, and autonomous agents let the expert work.** The single difference underneath is who decides the next step—the platform’s fixed loop, a workflow drawn in advance, the tenant’s own code, or a ready-made autonomous agent’s own judgment. Who decides determines what the platform can do to govern it, and which business it suits.



The cards above give the essence of each kind; Table 4 adds typical uses, who uses it, how it is governed and when it ships. Seen side by side, the selection logic becomes clear: first ask what you can hand to the platform (a prompt, a workflow, code, or a ready-made agent), then how deterministic the business needs the result to be, and finally whether the team can develop software.

Table 4 The four kinds compared

Dimension	T0 Q&A	T1 Workflow	T2 Code	T3 Autonomous
Who decides the next step	The platform's fixed loop	A workflow drawn in advance	The tenant's own code	A ready-made autonomous agent
What you hand over	Prompt and tool list	A workflow	Source code and dependencies	Task, environment, third-party agent
Development needed?	No	Business and risk define the workflow; developers supply tools	Yes	No coding, but skill in using agents
Determinism of results	Low—may differ each time	High—steps are provable	Depends on the code	Low—open-ended
Typical uses	Service desk, summaries, translation, classification	Verification, review, reporting, approval chains	Research, strategies, existing applications	Due diligence, data analysis, exploratory research
Not for	Risk decisions, money movement	Exploratory tasks, complex logic	Teams without developers	Direct high-risk actions; bulk low-value tasks
Who uses it	Business teams, self-service	Business and risk teams; developers supply tools	Business lines with developers; quant and research teams	Research, analysis and engineering teams
How it is governed	The loop is platform code	Static analysis before release	Exits only, not internals	Full-VM isolation and controlled exits
Delivery	Phase 1	Phase 2	Phase 1	Phase 3

Three mistakes recur when teams choose a kind, and they are worth naming here. First, using a Q&A agent for work that needs determinism: writing a risk decision or a compliance review as a prompt looks like the fastest way to ship, but a prompt cannot guarantee that a step is never skipped or that the same input never yields a different conclusion; that work belongs in a workflow agent. Second, letting an autonomous agent touch production directly: autonomous agents are for exploring and forming conclusions; the path they find should be distilled into a workflow, signed off, and promoted before it may place orders or change configuration. Third, treating the workflow language as a programming language: it has only six node types precisely so that it can be analyzed statically and signed off; forcing complex logic into it destroys that property, and the logic belongs in a code agent.

One more thing needs saying plainly: T0 to T3 is not a ladder from basic to advanced. The kinds express who owns the control flow and what it costs to govern it. The higher the risk, the more a business should converge on the workflow kind, not reach for more autonomy. A funds-handling process written as a signable workflow is far safer than the same process left to an autonomous model's judgment.

3.4 Choosing a kind

Figure 2 gives a selection path: ask three questions in order.

A rule of thumb: if it fits on a form, use T0; if it can be drawn as a workflow, use T1; if it needs code, use T2; if it cannot even be drawn, use T3.

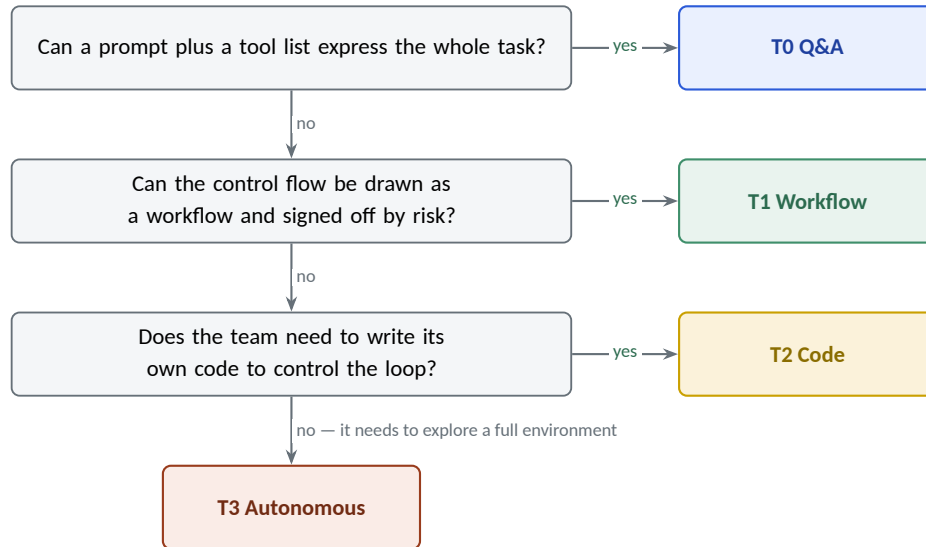


Figure 2 From business need to agent kind: three questions in order

3.5 Two questions that always come up

Is a workflow written in code a T1 or a T2? A T2. The test is who owns the loop: a graph written in Python or any other language runs inside the tenant’s process and is a program, not data, so the platform cannot analyze it before release. Only a workflow run by the platform’s executor, described in a declarative file, and analyzable and signable, is a T1.

What kind is a multi-agent system? Multi-agent is a topology between agents, not a new kind. The platform’s first-class hosted objects are three: agents, workflows and tool servers (MCP servers). One agent calling another takes the “register the callee as a tool” path and inherits every rule of tool governance (Section 10).

3.6 Moving between kinds

The kinds are connected by deliberate migration paths. T3 is for exploration: let a model find a workable path in a real environment. A high-risk, repeatedly executed path should not stay in T3; it should go through “explore → distil a draft workflow → compile and check → human sign-off” and become a T1 workflow, so that production always runs the signed diagram. In the other direction, a stable common loop that emerges in T2 can sink into the T0 harness, so that more teams reuse it by configuration alone.

4

Design Method: Principles, Invariants and Boundaries

The whole design reduces to three ideas. First, **put uncertain behavior inside a deterministic boundary**: a model’s output is unpredictable, but what it can reach, what it can do, and where its actions are recorded are deterministic; governance targets the boundary, not the model. Second, **control comes from authoritative state and governed exits, not from how much code you wrote**: whoever holds the record of what really happened and whoever guards the exit that every action must pass has control; everything else can be rented or reused. Third, **bind to a protocol, not to a framework**: agent frameworks keep changing; the platform freezes only the protocol between itself and the container, and any framework joins through an adapter.

The eight principles, the control formula, the ten invariants and the four build methods below are all elaborations of these three ideas: the principles say how decisions are made, the formula says where the boundary is cut, the invariants say what must never be violated, and the build methods say where each module comes from.

4.1 Eight design principles

Table 5 is the starting point from which every decision in the paper is derived. Each concrete design below can be traced back to one or more of them.

Table 5 Eight design principles

#	Principle	Engineering meaning
1	Bind to protocols, not frameworks	The platform freezes only protocol boundaries (HTTP, event streams, tool integration); a new framework is a new adapter, not a new platform
2	Separate control from capability	Event integrity, side-effect control and network boundaries are enforced by the platform and never rely on container code behaving; checkpoints and interruptibility rely on framework cooperation and are declared honestly at the level actually supported
3	Declare, then prove	A capability manifest is an application, not evidence; it takes effect only after a conformance test
4	Make state ownership explicit	Conversation, execution, environment and long-term memory each have an authoritative source, and recovery semantics are written into the contract
5	Generation may be imperfect; gates must be deterministic	Large models may help produce configuration and draft workflows, but deterministic checks—schema validation, policy engines, compilers—have the final word
6	Rich outside, minimal inside	The public API offers complete product semantics; the platform-container contract does only three things: be invoked, report liveness, be canceled
7	Own state and exits; rent compute	The enterprise keeps authoritative state and the exits where decisions are made, and leaves operating the stateful fleet to a vendor
8	Agents are first-class principals	An agent stands beside people and services, with an owner, a boundary, evidence, a history and a lifecycle

4.2 The control formula

The architecture’s judgment about what to build and what to outsource compresses into one formula:

Control = ownership of authoritative state + occupancy of the critical exits

Operational burden $\approx$ size of the stateful compute fleet you operate yourself

The first line says that whoever holds the authoritative record of what really happened, and whoever guards the exits that every action must pass, has control. The second says that the truly heavy operational burden lies in stateful compute fleets—scheduling, isolation, scaling, failure recovery. Put together, the conclusion is direct: keep authoritative state and the critical exits (they are light) and hand the compute fleet to a vendor (it is the heavy part). This is neither “build everything” nor “buy everything”; it is one cut placed according to where control and operational burden actually lie.

4.3 Ten system invariants

Invariants are not features. Features can be many or few and can be sequenced; invariants are constraints that no implementation and no enterprise instance may violate, and they form the acceptance checklist (Appendix C). Table 6 lists the ten, each with a way to verify it, so that none of them is a slogan.

Table 6 Ten system invariants and how each is verified

#	Invariant	Verification
1	No long-lived secret ever exists inside a container	Scan images, environment variables, processes and snapshots; credentials are fetched only by the tool gateway, briefly, per call
2	Side effects pass only through the governed exits	Negative network tests: direct calls from a container to a model, a tool or the internet must fail
3	Tool calls are authoritative only as recorded by the tool gateway	Compare container-reported events with gateway events; audit trusts only the gateway
4	The ledger stays inside the enterprise boundary, append-only, hash-chained	Modifying any historical event breaks the chain check
5	Every run has a hard ceiling	Exceeding any of steps, calls, model usage, duration or spend terminates the run
6	Tokens are separated by interface	A run token used against the management API, or a platform token used against a runtime exit, is rejected
7	User identity is verified against the identity provider	A forged “on behalf of” string cannot enter the identity chain
8	Shutdown does not depend on the cloud vendor	With the vendor’s API simulated as unavailable, revoking issuance and gateway refusal still strip the agent of the ability to act
9	Event sequence numbers are assigned by the platform, recorded before delivery	After a disconnect, a client resumes continuously from the ledger by sequence number alone
10	Administrative and agent actions share one ledger	Release, authorization, suspension and key rotation can be replayed in relation to run events

4.4 Four ways to build

Every module is built in one of four ways: built in-house, reused from an existing enterprise system, adopted as an open standard, or rented from a cloud vendor. Table 7 gives the scope of each and the exit requirement it carries.

One rule decides which cloud services may be rented: **rent the muscle, never the brain or the keys**. A component is rentable only when it satisfies three conditions at once: it holds no decision rights over what may or may not happen; it does not host the enterprise’s only authoritative data (the ledger, credentials, identities); and it follows an industry standard, so that the cost of leaving is close to zero. Measured against that rule, a vendor’s full product line yields exactly four rentable items: runtime, sandbox, object storage and the observability dashboard. The dashboard passes because it only observes, never touches the ledger, and receives data through the OpenTelemetry open standard, so that switching means changing a destination address. A vendor’s identity service, gateways, credential hosting, knowledge base and memory service all fail the test, because each either holds decision rights or hosts authoritative data.

Table 7 Four ways to build

Method	Scope	Why	Exit requirement
Build	Identity, authorization, the ledger, the task engine, the cloud adapter	These constitute control itself, and no complete product exists for them	Metadata and management APIs depend on no vendor
Reuse	Identity provider, model gateway, tool gateway, CI, finance systems	Enterprises usually already have mature versions; rebuilding is costly and fragments existing processes	Incremental integration through the platform's standard interfaces
Open standard	PostgreSQL, OpenTelemetry, event and object-storage semantics	Mature ecosystems lower migration and operations barriers	Data exportable, formats versioned
Rent	Runtime, sandbox, dashboards, object storage	Elasticity, usage-based pricing, vendor economies of scale	Holds no decision rights, holds no unique authoritative data, has a standard exit

5

Related Work and Where This Paper Stands

By September 2026 the enterprise agent-infrastructure market has converged on several clear patterns. This paper builds on that consensus rather than starting from scratch.

From vendors' public documentation: AWS's AgentCore splits runtime, identity, gateway, memory and observability into composable services; Anthropic's Managed Agents provide stateful sessions and a durable event history; OpenAI's Agents SDK covers sessions, sandboxes, tools, credential vaults, guardrails and tracing; Google's Agent Platform offers a managed runtime, sessions, a memory bank, code execution, access control and observability; Microsoft's Entra Agent ID models the agent as an independent identity principal with a sponsor and lifecycle management [1, 2, 3, 4, 5].

In execution reliability and isolation, LangSmith Deployment and Temporal provide durable execution, event history, interrupt and resume, and long-running orchestration; Tencent Cloud's open-source Cube-Sandbox represents the infrastructure route of hosting agent sandboxes on micro-VMs with snapshots and controlled networking [6, 7, 8].

In April 2026, within the same week, Anthropic and OpenAI each released structurally identical architectures: a layer responsible for control flow and model calls, explicitly separated from a layer responsible for isolated execution, with credentials never entering the execution sandbox. "Separate the control plane from the execution plane, keep credentials out of the sandbox, treat gateway events as authoritative" is therefore an industry consensus rather than one vendor's choice.

This paper's position is to close the **enterprise control boundary** between those capabilities. Each of the prior systems solves one segment of the problem; assembling them into a platform the enterprise fully controls, can migrate as a whole, and can answer to auditors for still requires a set of enterprise-side contracts and mechanisms. The paper's additions are: an agent capability described along four orthogonal axes and certified by test; a minimal container contract with platform-assigned event sequencing; a permission-intersection rule covering scheduled principals, user delegation and structural denials; a three-stage shutdown path independent of the vendor's management API; effectively-once semantics built from checkpoints, a tool idempotency ledger and archived results; and a hash-chained enterprise ledger that supports compliant deletion and reconciles against vendor bills.

Table 8 Capabilities the industry already provides, and what this paper does not claim

Existing capability	Representative prior work	Not claimed as new here
Managed runtime and elastic scaling	AgentCore Runtime, Managed Agents, Google Agent Runtime	Deploying agents to a managed compute environment
Identity, credentials and tool governance	AgentCore Identity and Gateway, Entra Agent ID, OpenAI Vaults	Independent agent identity, credentials kept out of business code, tool calls constrained by policy
Sessions, memory and observability	Google Sessions and Memory Bank, OpenAI tracing, LangSmith	Persisting session state, tracing call chains, analyzing run quality
Durable execution and isolated sandboxes	Temporal, LangGraph and LangSmith, CubeSandbox, cloud code execution	Recovering after failure, waiting for human input, isolated code execution

PART II

Architecture and Runtime Contract

From the control boundary into engineering: the six-layer architecture, the four-axis manifest, the three interface layers, runtime state, and how each kind of agent fulfills the contract.

6 Overall Architecture

The architecture rests on one asymmetry: the control plane is owned by the enterprise and never leaves it; the execution plane is rented and can be replaced. This section shows the layers, the partitions inside the control plane, and every module in one picture.

6.1 The six-layer view

From the people who use it down to the cloud resources it runs on, the platform has six layers (Figure 3). Three lines matter when reading the figure: enterprise systems are not submodules of the platform but are reused through explicit interfaces; the platform's own control plane is one whole, owned by the enterprise; and a stable contract separates the control plane from cloud execution resources, so everything below the contract can be swapped.

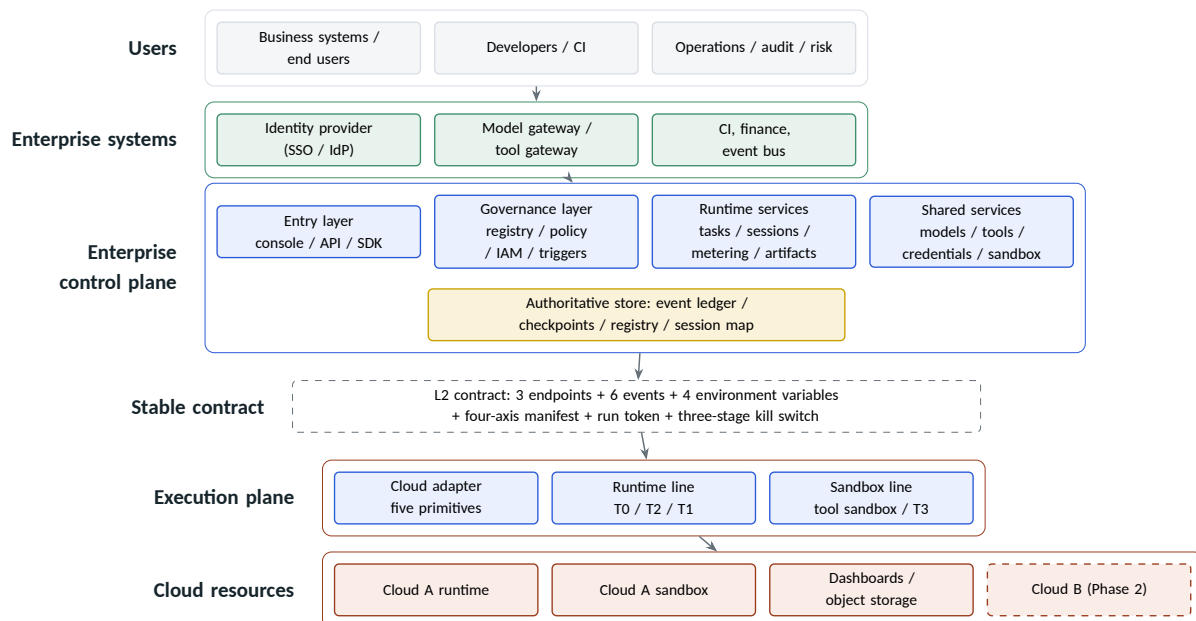


Figure 3 The six-layer boundary of the platform. Blue: enterprise-owned platform modules; green: reused enterprise systems; yellow: open standards; orange: rented cloud resources

Layer by layer, from the top:

Users Four roles, each with its own entrance: business systems call agents through the unified API; end users reach agents through business front ends; developers onboard, build and release through the SDK and command line; operations, audit and risk staff inspect, suspend and replay through the admin console.

Enterprise systems The identity provider, the model gateway, the tool gateway, CI, finance systems and the event bus. The platform does not rebuild any of them; it integrates through standard interfaces (Section 19).

Enterprise control plane All of the platform's own logic, in five partitions (next subsection). This layer is where control lives, and the enterprise owns it.

Stable contract The agreement between the platform and every agent container. It is frozen on day one and only ever extended, so that everything below it can be replaced freely and nothing above it needs rework.

Execution plane Where agents actually run. The cloud adapter translates platform instructions into vendor APIs; two product lines host long-running agents and ephemeral isolated computation respectively. The shells and the adapter belong to the platform; the machines are rented.

Cloud resources Rented machines, sandboxes, storage and dashboards. Vendor names appear only in this layer.

6.2 The five partitions of the control plane

Modules inside the control plane are grouped into five partitions, each with an explicit criterion (Table 9) that tells any new module where it belongs.

Table 9 The five partitions of the control plane and their criteria

Partition	Criterion	Core modules
Entry layer	The entrances through which people, CI and business systems reach the platform	Admin console, unified API (L1), SDK and CLI
Governance layer	Low-frequency governance: what may run, who may do what, when it is woken	Registry and release, policy and authorization, agent IAM, scheduled and event triggers, approval console
Runtime services	The high-frequency path every run passes through	Task engine, session service, metering and reconciliation, files and artifacts, observability, interrupt and resume
Shared services	The exits that agents running in the cloud must pass to reach enterprise capabilities	Model gateway, tool gateway, credential vault, sandbox service
Authoritative store	The enterprise-side source of truth for all control and runtime facts	PostgreSQL event ledger, checkpoints, registry, session map and hash chain

Two points deserve explanation. First, why the entry layer and the shared-services layer are kept apart: traffic in the entry layer flows *in* (people and business systems calling the platform); traffic in the shared-services layer flows *back* from the cloud (running agents calling models and tools). Opposite directions, different callers, different authentication—mixing them would let one set of governance rules contaminate the other. Second, why the authoritative store is its own layer: it is not a module's private database but the source of truth every module depends on, and drawing it separately states plainly that this data must be in the enterprise's hands.

6.3 The complete architecture

Figure 4 draws every module on one page. It shows the target architecture in full, without distinguishing delivery phases (phase scope is in Section 20); color indicates the build method. Read it from top to bottom: users enter through the entry layer; existing enterprise systems are reused around the platform; the platform’s own control plane has five partitions; one stable container contract separates the control plane from the execution plane; in the execution plane the shells are the platform’s and the machines are rented; vendor names appear only in the bottom layer. Later sections zoom into each region: the entry layer in Figure 8, the governance layer in Figure 5, runtime services in Figure 11, shared services in Figure 18, the authoritative store in Figure 21, the execution plane and cloud resources in Figure 22, and enterprise systems in Figure 23.

Table 10 gives a one-line responsibility and the build method for each module. Phase information appears only in this table and in Section 20; the architecture figures show the target as a whole.

6.4 The vendor boundary rule

Vendor names may appear only in the cloud-resources layer. Wherever the control plane or the execution plane needs a vendor SDK, it must be wrapped by the cloud adapter and must not reach the task engine, identity, metering or the public API. The value of this rule is that switching vendors is confined to a single module.

7

Capability Manifest and Admission

This section corresponds to the governance layer in the complete architecture. Its modules are the low-frequency governance actions (Figure 5): registry and release decide *what may run*; policy, authorization and agent IAM decide *who may do what*; scheduled and event triggers decide *when an agent is woken*; the approval console handles high-risk operations that need a person. Together they answer one question—on what grounds is an agent allowed to run?—and the answer is the capability manifest and the admission rules of this section.

7.1 Why not bind everything to the kind

Section 3 divided agents into four kinds by who decides the control flow. A natural next step would be to bind isolation level, side-effect scope and recovery capability to the kind as well: T0 means shared pool plus read-only, T2 means dedicated container plus governed writes. Inside one enterprise this works; as a general platform it keeps running into exceptions—a code agent that needs full-machine sandboxing, a Q&A agent that needs a browser tool, a workflow agent that needs dedicated compute. Every exception brings a proposal for a “fifth kind”.

The approach here is to split those properties into four independent axes, written into each agent’s **capability manifest**. The kind is merely a set of default values on the four axes; the policy engine and the audit rules operate on the axes, never on the kind’s name. A new scenario is a new combination, not a new kind.

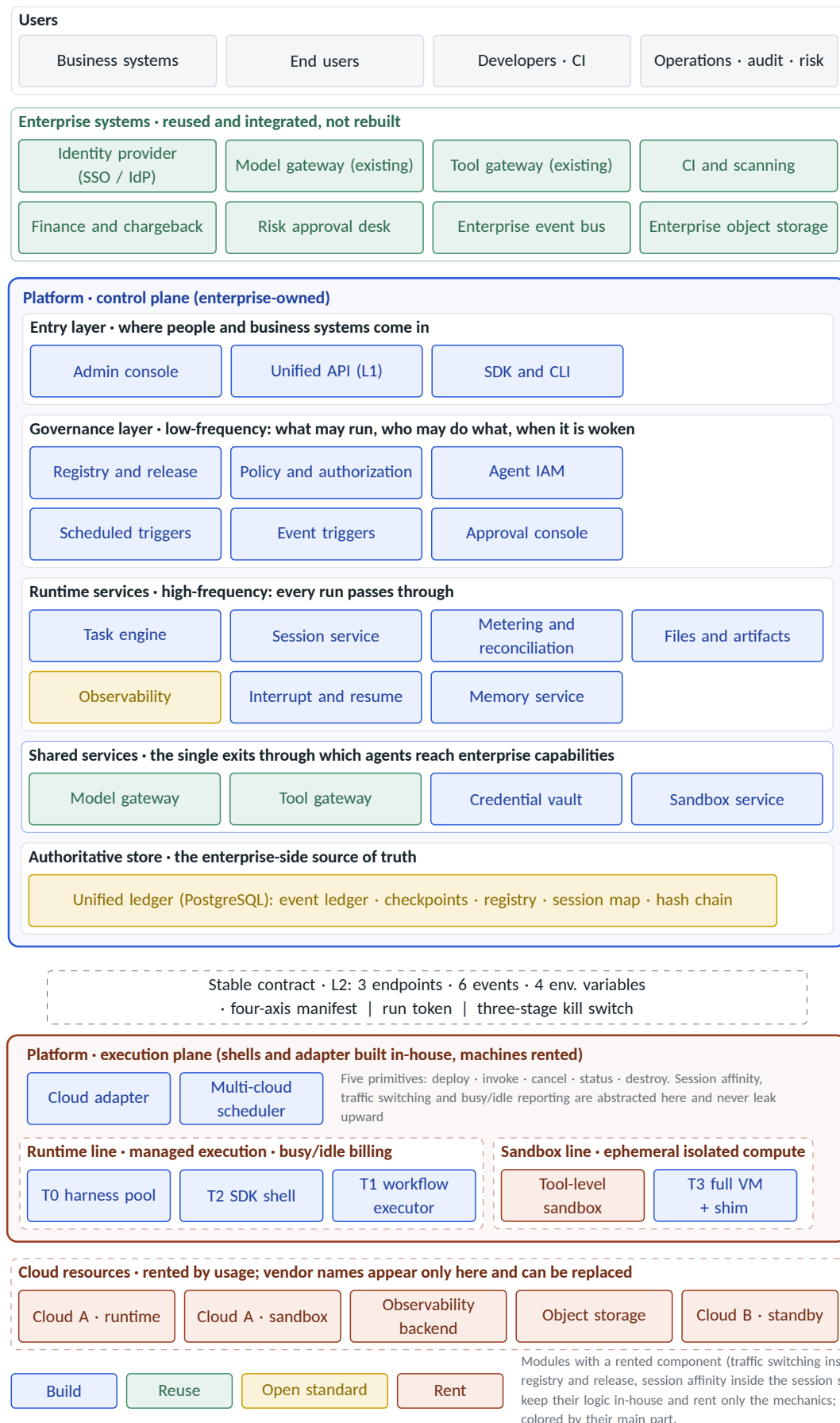
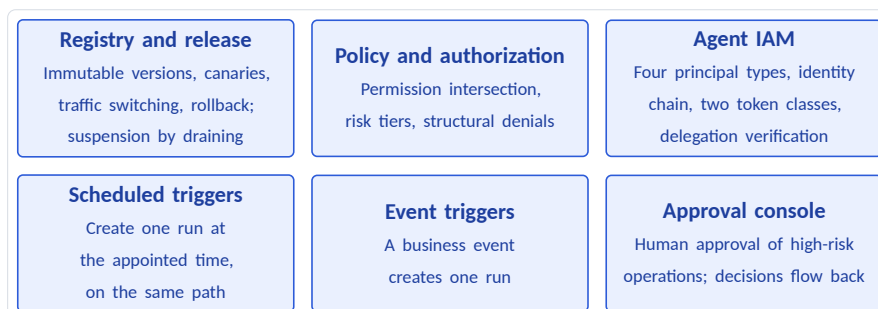


Figure 4 The complete architecture: four layers and every module, colored by build method (phases in Section 20)

Table 10 Module build matrix

Partition	Module	Method	Phase	Responsibility
Entry	Admin console	Build	1	Inspect, manage, replay, one-click suspend; every action goes to the ledger
	Unified API (L1)	Build	1	Threads, runs, events, files; streaming output and resumable reads
	SDK and CLI	Build + Rent	1	Onboarding with few changes, cloud build, release, conformance checks
Governance	Registry and release	Build + Rent	1	Immutable versions, canaries, traffic switching, rollback, suspension
	Policy and authorization	Build	1	Permission intersection, allow-lists, structural denials, tool risk tiers
	Agent IAM	Build	1	Principals, owners, identity chain, two token classes, delegation verification
	Triggers and approval	Build	1–2	Scheduled triggers first; event triggers and the approval console in Phase 2
Runtime	Task engine and sessions	Build + Rent	1	Run state machine, circuit breaker, retries, thread-to-session mapping
	Metering and reconciliation	Build	1	Internal allocation book, vendor book, reconciliation book
	Files and artifacts	Build + Rent	1	Reference management, presigned direct transfer, lifecycle
	Observability and recovery	Open standard + Rent	1–2	OpenTelemetry instrumentation first; interrupt mechanics and memory in Phase 2
Shared	Gateways, vault, sandbox	Reuse + Build	1	Single exits for side effects, credential injection, isolated execution
Store	Unified ledger	Open standard	1	Events, checkpoints, registry, mapping; point-in-time recovery
Execution	Cloud adapter, T0, T2	Build	1	Five primitives, harness pool, code-agent SDK shell
	T1, T3, multi-cloud	Build	2–3	Workflow executor, autonomous-agent shim, cross-cloud replay
Cloud	Runtime, sandbox, storage	Rent	1	Elastic machines, isolated environments, dashboards and object storage

**Figure 5** The six modules of the governance layer (the governance region of the complete architecture)

7.2 The four axes

Table 11 The four axes of the capability manifest

Axis	Values	Meaning and who decides
Loop owner loop_owner	platform harness / platform workflow executor / tenant process / third-party process	Who implements the loop, and therefore who fulfills the container contract; decided by the definition carrier
Isolation isolation	shared pool / dedicated container / full VM	Derived from risk and tool reach; the platform may raise it, the tenant may not lower it
Side effects side_effect	read-only / governed writes / sandboxed / external	Enforced by network policy and the tool gateway, not by declaration
Recovery recovery	checkpoint: per step / per session / none; interrupt: cold / hot / none	Declared, then certified by conformance test



The T0–T3 marks are each kind's default values; admission checks the table of legal combinations by axis, not the kind's name.

Figure 6 The four axes of the capability manifest, with each kind's defaults

Each axis has its own arbiter, and that is the point of the design. The loop owner is fixed objectively by the definition carrier, so a tenant cannot misstate it. Isolation is derived by the platform from risk, and a tenant can only accept a stricter level. Side-effect scope is not read from the declaration but from what network policy and the tool gateway actually allow. Recovery capability may be declared, but it counts only after a conformance test. On none of the four axes does “whatever the tenant says” decide.

7.3 Legal combinations and admission rules

Admission is no longer hard-coded by kind; it consults a table of legal combinations. The Phase 1 rules are:

- An agent whose side-effect scope is “external” must run in a dedicated container or stronger and must checkpoint after every step. Reason: an agent that can cause irreversible effects on external systems

needs both stricter isolation and the ability to resume from the last safe point after a failure rather than restarting from the beginning.

- An agent without checkpoints may hold only a read-only tool set, and the platform’s automatic retry is switched off for it. Reason: without a checkpoint there is no way to know whether a side effect occurred before the failure, so an automatic retry might perform an action twice.
- Until the interrupt mechanics ship (Phase 2), any manifest that declares interrupt capability is refused at admission. Reason: the platform cannot yet handle the interrupt requests it would emit; it is better to stop it before release than to fail at runtime.
- High-risk side-effect paths (money movement, for example) may be carried only by the platform’s workflow executor or a tenant process, and require per-step checkpoints, cold interrupt (release the machine and recover from the checkpoint), and human approval. Reason: such paths must be statically analyzable or externally constrainable, and must be able to stop and wait for a person.
- Isolation is raised automatically by the platform according to tool risk; tenants cannot request a lower level.

The rules themselves can evolve: security policy, runtime implementation and product presets change independently, and no new kind of agent requires a copy of the infrastructure.

7.4 Why resident agents are not supported

One request recurs: “keep an agent running to watch the market” or “continuously listen to a queue”. The platform deliberately has no “resident” kind. A resident process breaks a basic assumption—that the **run** is the unit of metering, audit, cancellation and replay. A process that never ends has no cost that can be attributed to one action, no start and end for audit, and no safe point for cancellation.

The right model is to turn “continuous listening” into a scheduled or event trigger that creates one bounded run per firing, so that each action is separately billed, separately authorized and separately accountable. Scheduled triggers ship in Phase 1, event triggers in Phase 2.

8 Three Interface Layers

The platform’s interfaces follow one rule: complete on the outside, minimal on the inside. This section defines the three layers and the contract that must never be reworked.

8.1 What each layer does

The platform has three interface layers (Figure 7): the unified API facing users (L1), the container contract between the platform and each container (L2), and the service exits through which containers reach enterprise capabilities (L3). The three are designed in deliberately opposite directions: L1 is as rich as possible, so business systems get product semantics directly; L2 is as thin as possible, so an agent written in any framework can satisfy it easily; L3 is a set of single exits, so that however a container is written it cannot bypass governance.

One simple rule connects the layers: every data-path operation on L1 ultimately reaches the container as one of only three actions—invoke it, probe whether it is alive, cancel it. Anything that requires *remembering* something (conversation history, run progress, approval tickets) lives on the platform side, never in the

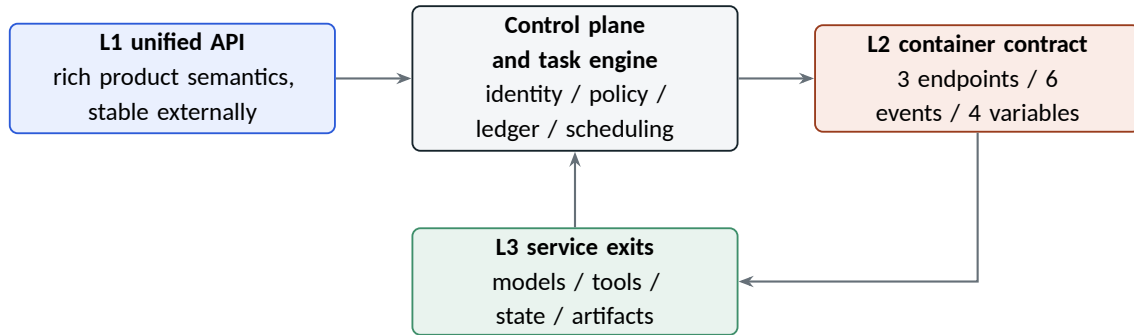


Figure 7 Separation of responsibility among L1, L2 and L3

container. The rule answers the recurring question of whether a feature belongs in the platform or in the container: the container does only what needs no memory.

8.2 L1: the unified API for users

L1 exposes resources that a business can understand and hides containers and vendors entirely (Table 12). It is the one of the three entry-layer modules that faces business systems; the other two—the SDK and CLI, and the admin console—face developers and operations staff respectively (Figure 8). Requests from all three entrances converge on the same control plane and the same ledger.

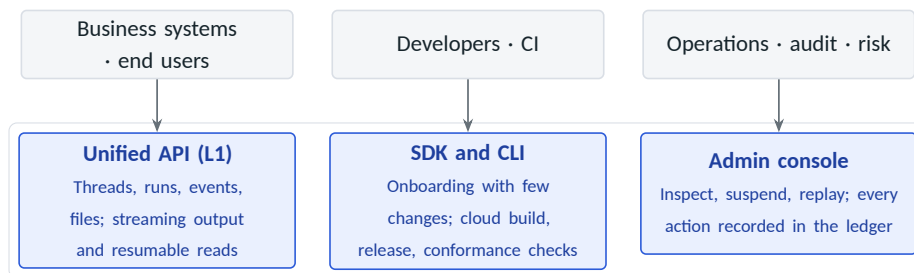


Figure 8 The entry layer: one entrance for each class of user (the entry region of the complete architecture)

L1 has several cross-cutting constraints, each of which prevents a specific class of incident:

- **Idempotent creation.** POST/runs accepts an Idempotency-Key; a repeat from the same caller with the same key within 24 hours returns the same run. This prevents a business system that times out and retries from creating two runs, with side effects and cost doubled.
- **Mandatory tenant filtering.** Every read interface filters by tenant as a mandatory condition, never as an optional parameter. This prevents a tenant that guesses someone else's run ID from reading their events.
- **Platform tokens only.** L1 accepts only the platform tokens issued to people, CI and business services, and rejects the run tokens held by containers (Section 14). This prevents code inside a container from using its run token to call the management API and change authorizations.
- **Trace propagation.** Every request carries a standard trace identifier (W3C traceparent) that propagates across the whole chain.

Table 12 Resource groups of the L1 unified API

Resource group	Core operations	Key semantics
Threads	Create, query, fetch history, delete	Long-lived conversation records; history is served by the platform, the container is not involved
Sessions	Query, end explicitly	Short-lived isolated execution environments, usually managed implicitly by the platform
Runs	Start, query status, cancel, resume (Phase 2), retry	Synchronous streaming or asynchronous; the run is the subject of every state machine
Events	Read from a sequence number	Clients read events only from the enterprise ledger, never directly from the container
Interrupts	Pending list, structured tickets, decisions	Phase 2; a decision reuses “resume” and enters the same state machine
Artifacts and files	Declare, upload, download, delete	Large objects travel by reference and presigned URLs, never through the API body
Governance resources	Agents, versions, tools, triggers, usage	Registration, release, authorization, triggers and usage queries

8.3 L2: the minimal container contract that must never be reworked

L2 is the platform’s most important long-term asset. It carries no product convenience; it expresses only the minimum obligation that any agent container must fulfill. The whole contract is three endpoints, six events, four environment variables and one capability manifest.

```
POST /invoke {run_id, thread_id, input, resume?, context: read-only}
    -> server-sent event stream (SSE)

GET /ping -> Healthy | HealthyBusy

POST /cancel {run_id} -> 202
```

The three endpoints mean: `/invoke` hands a run to the container, which returns its results progressively as an event stream; `/ping` probes whether the container is still alive—“busy” is only a hint, and the authoritative busy/idle judgment is made on the platform side (Section 18); `/cancel` asks the container to exit at its next safe point, and if it has not done so within the grace period, the cloud adapter destroys it.

The six events reduce the internal differences among frameworks to one shared semantics (Table 13). Any framework that can translate its own execution into these six events can be hosted.

The four environment variables deserve a word. They are how the platform tells a container “where the models are, where the tools are, where checkpoints go, where artifacts go”. The container holds no secret at all—only these four addresses. The short-lived run token bound to the current run is not placed in an environment variable; it is injected with each invocation’s read-only context, valid for minutes, then void. That is how Invariant 1 (no long-lived secret inside a container) is implemented.

8.4 Persisting and distributing events

Events produced by a container are recorded by the platform first and distributed to clients second. This serves three purposes: a client that disconnects can resume from the ledger by sequence number; events

Table 13 Composition of the L2 contract

Component	Content	Purpose
3 endpoints	invoke, ping, cancel	Be invoked, report liveness, be canceled
6 events	message.delta (output fragment), tool.call (tool call), checkpoint, interrupt (request for input), artifact (reference), run.status (state and terminal status)	Reduce framework differences to one event semantics
4 environment variables	MODEL_GATEWAY_URL, TOOL_HUB_URL, CHECKPOINT_STORE, ARTIFACT_STORE	Inject only the addresses of platform capabilities, never a long-lived secret
1 manifest	Four axes, streaming flag, maximum run duration, harness version	Declare capability, limits and the harness version

remain replayable after the container has exited; and event order is decided by the platform, so a container cannot forge it.

“Record first” must not, however, be read as one synchronous database write per output fragment—that would put a disk write on the streaming path of every token and visibly degrade the streaming experience. Different events therefore get different persistence levels (Table 14): tool calls, checkpoints, artifacts and terminal status are fact or control events and must be committed to the main ledger before distribution; output fragments serve the streaming experience, so they receive a sequence number, are written to a short-retention stream table, are distributed immediately, and are merged into message records in batches.

Table 14 Persistence levels for the two classes of event

Event class	Condition before distribution	Semantics on failure
Fact and control events	Committed to the main event table	Fully resumable and auditable; loss is not accepted
Output fragments message.delta	Received by the short-retention stream table and given a sequence number	Weak persistence allowed; at most the fragments not yet merged may be lost

The container supplies an event_id for de-duplication but cannot choose the sequence number; the platform assigns it on receipt. If the container-to-platform link breaks, the run enters the failed state marked retryable, and whether it is actually retried is decided by the rules in Section 11.

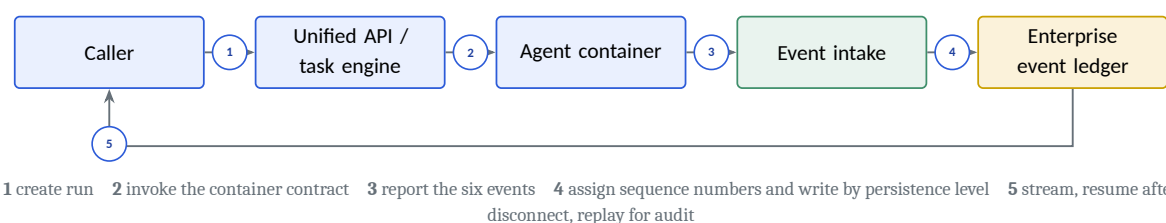


Figure 9 Record first, then distribute: fact events are durably written before delivery; output fragments go through a short-retention stream table to keep streaming latency low

8.5 How the contract evolves

The format of the six events is frozen in Phase 1, but frozen does not mean unchangeable. The evolution rule is “add, never alter”: new fields must be optional; every event carries a `schema_version`; and the platform reads at least two versions for a period. The interrupt event and the resume field are defined in Phase 1 and implemented in Phase 2, so that Phase 2 does not change the contract of any container already in production.

8.6 L3: the exits through which containers reach enterprise capabilities

L3 consists of the model gateway, the tool gateway, the checkpoint store, the artifact service and (in Phase 2) the memory service. A running container can reach external capabilities only through L3: the platform injects the addresses of these services as environment variables, and the container authenticates with a run token valid for minutes. The container holds neither a management-API token nor any long-lived tool credential. Section 15 describes this layer in detail.

9

One Request, End to End

The preceding sections covered the architecture, the capability manifest and the interfaces separately. This section strings them together on one concrete example: a business system asks an agent to read some business data and produce a report file. Nothing in the walk-through depends on a particular agent framework or cloud vendor.

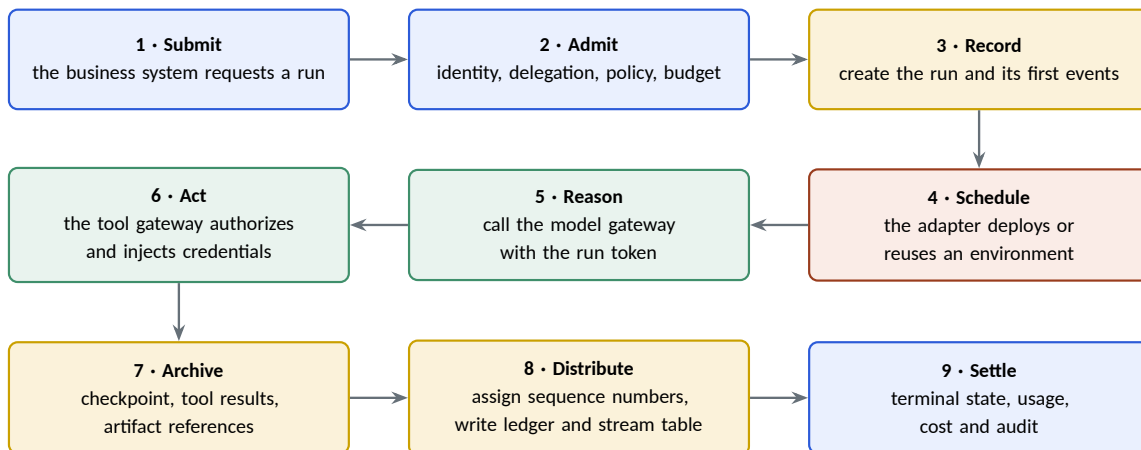


Figure 10 One run in nine steps, from business request to settled facts

- 1. Submit.** The business system calls `POST /runs` with its own platform token, an idempotency key, and—if the call is made on behalf of a user—an on-behalf-of token issued by the identity provider.
- 2. Admit.** The unified API resolves the tenant from the authenticated identity, verifies the signature of the delegation token, and checks that the agent version is available, that the tool authorizations are valid, that no structural denial is touched, and that the budget is not exceeded. Any failure rejects the request here, recorded in the ledger as a *policy denial*, not a system error.
- 3. Record.** The task engine creates the run under the idempotency key (a repeat returns the same run) and writes the admission result and initial state to the enterprise ledger. From this moment the run has a sequence number, an identity chain and a budget ceiling.

4. **Schedule.** The scheduler chooses execution resources from the capability manifest, deploys a new environment or reuses an existing one through the cloud adapter, and calls the container's `/invoke`.
5. **Reason.** The container holds only a short-lived token bound to this run. When it needs a model, the request goes through the model gateway, which checks the model allow-list, deducts the usage budget and records the token count of the call.
6. **Act.** The agent decides to call a tool that reads business data. The tool gateway recomputes the agent's effective permission at that moment, checks the idempotency key, fetches the required credential briefly from the credential vault and calls the target system on the agent's behalf. The container never touches the credential.
7. **Archive.** Tool results, model results and the checkpoint taken before the side effect are archived. The generated report is stored in object storage; the ledger keeps only a content digest and a reference.
8. **Distribute.** The event intake de-duplicates, assigns sequence numbers, writes fact events to the main ledger and output fragments to the short-retention stream table, and streams to the client. A disconnected client resumes by sequence number.
9. **Settle.** When the run reaches a terminal state, the platform totals model, tool, runtime, sandbox and storage usage and produces the internal allocation, vendor reconciliation and audit views.

Several things are visible in this walk-through. The container never obtained a secret. Every external action—calling a model, calling a tool—passed an exit the platform can record and refuse. The order and authenticity of events were decided by the platform, not the container. And the cost was attributed to a specific tenant and run the moment the run ended. Those four observations are, respectively, controllable, auditable, observable and meterable.

10 Runtime Semantics

State, channels and execution facts: this section fixes the vocabulary that the runtime services share. This section and the next correspond to the runtime-services partition of the complete architecture (Figure 11)—the high-frequency path every run passes through. The task engine and the session service track where a run has got to and which conversation it belongs to; metering and reconciliation track what it cost; files and artifacts track what it produced; observability tracks whether the system is healthy; interrupt and resume and the memory service are two capabilities added in Phase 2. This section first covers the runtime semantics these modules all rely on: how state is divided, how many channels exist, and how a run changes state.

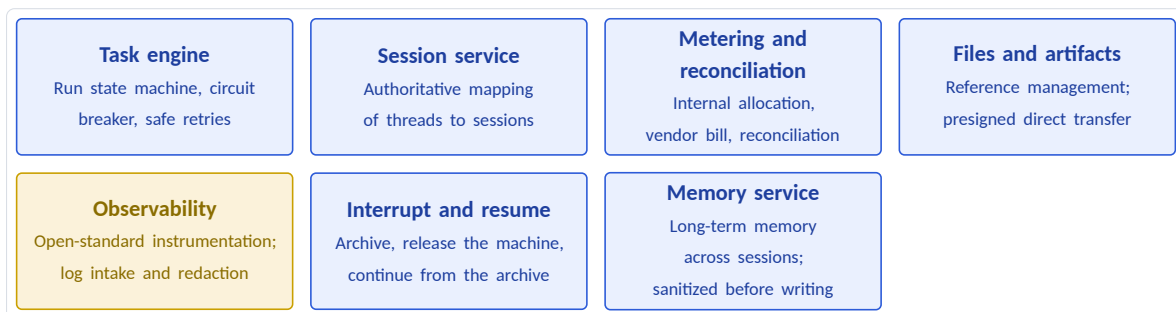


Figure 11 The seven modules of runtime services (the runtime-services region of the complete architecture)

10.1 Three concepts that must not be merged: session, thread, run

The platform describes runtime state with three concepts that are often confused but govern different things (Table 15). A plain analogy: a **session** is like a private room opened for a visit and closed afterwards; a **thread** is like the complete record of one customer's dealings with the enterprise across many visits; a **run** is like one specific piece of business, with a start, a result and a cost.

Table 15 Session, thread and run

	Session	Thread	Run
What it is	A lease on an isolated execution environment	The history of a conversation	One execution with a terminal state
What it governs	Isolation, resources, in-memory state	Continuity of context across turns	Progress, events, retries, cost
Lifetime	Occupies a machine while active, released when idle	Long-lived, spanning many sessions	Minutes to hours
Authoritative source	The platform's session map plus the vendor's lease	The enterprise session service	The task engine and the event ledger

A thread may span many sessions: each active period is bound to one session, the session is released when idle, and the next active period opens a new one. The mapping between threads and sessions is authoritative data held by the enterprise; the mechanical implementation of “route the same user's requests to the same machine” (session affinity) can be rented from the vendor.

10.2 Five action channels and one set of state services

A running agent can affect the outside world through exactly five channels (Figure 12). Outside those five is a wall made of network policy and sandboxes. As long as the platform guards the five channels, side effects, identity, cost and events all enter one governance chain even when the code inside the container is completely untrusted. State and data services (checkpoints, artifacts, memory) support the channels but are not action channels themselves, because they do not affect the outside world.

Table 16 The five action channels and their governance

Interaction	Only channel	Governance on the channel
Being called	The platform's invoke interface	Authentication, rate limits, circuit breaker, placement; container ports are not exposed publicly
Calling a model	The model gateway	Data classification, model allow-list, usage budget, metering, blocklist
Acting on external systems	The tool gateway	Permission intersection, risk tiers, parameter-level controls, idempotency, authoritative events
Asking a person	An interrupt ticket (Phase 2)	Routing by type, timeouts, approver and decision recorded in the event stream
Calling another agent	The callee registered as a tool	Tool governance applies; delegation can only narrow permissions
State and data	Checkpoints, artifacts, files, memory	Tenant isolation, expiry, transfer by reference, sanitization before writing; not an action channel

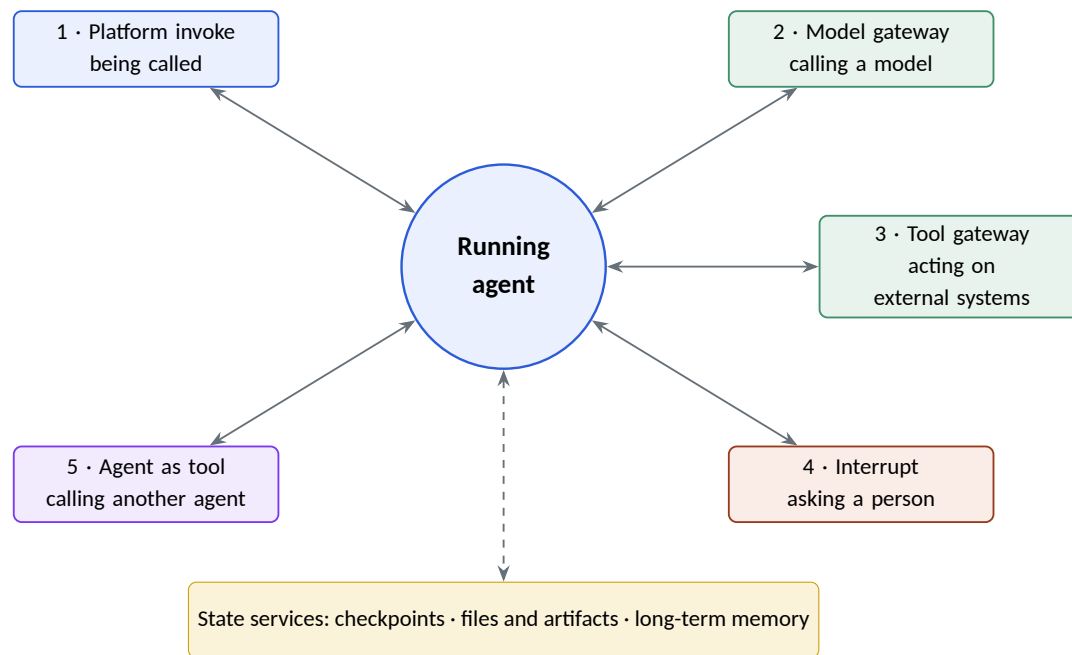


Figure 12 Five action channels form a closed governance boundary; state services carry recovery and data separately

This closed boundary constrains recognizable interfaces, networks and platform state channels. Covert channels—DNS, timing signals, or influencing a later input through artifact content—are outside the threat model of this paper and must be handled separately with egress proxies, content sanitization, rate limiting and independent security assessment.

10.3 The run state machine

A run's state changes are simple (Figure 13): queued, running, and then one of three terminal states—completed, failed or canceled. Phase 2 adds an “input required” state for interrupt and resume.

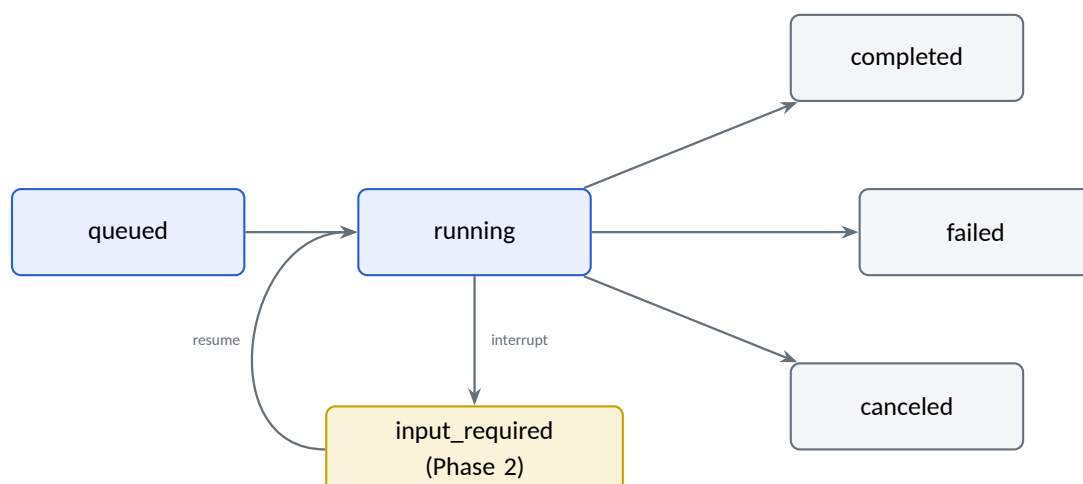


Figure 13 The run state machine; policy denials and budget overruns use their own error classes

The failed state carries an error class with at least four values: retryable, terminal, policy denied, budget exceeded. “Policy denied” deserves emphasis: an agent stopped by risk control is not a platform fault but evidence that control is working. It must be distinguished from genuine faults in monitoring metrics, in audit records and in what the caller sees; otherwise operations staff will treat normal interventions as incidents to be “fixed”.

10.4 Who owns which state

Principle 4 in Section 4 requires state ownership to be explicit. Table 17 gives the assignment; recovery follows it.

Table 17 Authoritative ownership of the four kinds of state and what recovery means

State	Authoritative owner	Meaning at recovery
Conversation state	The session service and the thread	Provides continuity of history; not the same as resuming execution
Execution state	The checkpoint	Determines the safe point to continue from
Environment state	The session's file system and artifacts	Needs references, snapshots or rebuilding
Long-term memory	The memory service (Phase 2)	Not part of execution recovery; sanitized before writing

11 Reliability and Safety Controls

This section covers four mechanisms: the circuit breaker, idempotency, recovery, and shutdown. Together they answer one question: when an agent runs away, repeats itself, gets stuck, or must be stopped at once, what bounds the damage?

11.1 The circuit breaker: a hard ceiling on loss

The task engine sets hard ceilings on every run: number of steps, number of tool calls, model usage, wall-clock duration and estimated spend. Exceeding any one of them moves the run immediately to the “budget exceeded” terminal state. At the tenant level there are ceilings on concurrency and daily spend, with overridable defaults; the shared pool has both pool-level and tenant-level concurrency ceilings, so that one tenant’s burst cannot crowd out the others.

The circuit breaker and quota management are two different things. Quota management—screens, approval workflows, periodic policies—can be built in Phase 2; the circuit breaker is a fuse and must exist in Phase 1. The reason is blunt: without it, the platform discovers its first runaway loop only after the fact, when the metering system can explain how much money was lost but could not have prevented it. The breaker costs almost nothing to build—a few counters and comparisons—and it is the mechanism most likely to be needed in the first week of production.

11.2 Two kinds of idempotency, kept apart

“Idempotent” means that repeating an operation produces no additional effect. The platform needs two kinds, with different keys and different purposes (Table 18).

Automatic retry is capped at three attempts by default, with increasing intervals. Whether a failure may be retried automatically is decided not by what the container claims but by the tool gateway’s idempotency ledger: a failure is retryable only if no unconfirmed side effect has occurred since the last checkpoint. Agents without checkpoints have automatic retry switched off by default.

Table 18 Two kinds of idempotency

Kind	Key	Problem it solves
Creation idempotency	An Idempotency-Key scoped to the caller	A network retry must not create multiple runs
Side-effect idempotency	run_id + step_id + attempt	Recovery or retry must not perform a real action a second time

11.3 Effectively-once side effects

A distributed system cannot guarantee, from a single network request, that a real-world action happens exactly once. The architecture composes an equivalent guarantee from three things:

checkpoint before the effect + tool-gateway idempotency ledger + archived results
 $\Rightarrow$ effectively once

The first ensures there is a point to return to after a failure; the second ensures that, having returned, an action already performed is not performed again; the third ensures that the model's output is archived too—recovery does not re-sample the model, because otherwise the same run might take a different branch after recovery.

11.4 The three-stage kill switch

Stopping a rogue agent cannot depend on the cloud vendor's management API happening to be available at that moment. The shutdown path therefore has three stages (Figure 14):

- 1. Stop issuing.** Disable the agent in the registry; the platform issues it no new run tokens.
- 2. Cut its hands.** The model gateway and the tool gateway reject the tokens it already holds, by agent ID. From this point it can call no model and no tool, and therefore cannot cause any further external side effect.
- 3. Destroy the instance.** The cloud adapter calls the vendor's API to reclaim the machine.

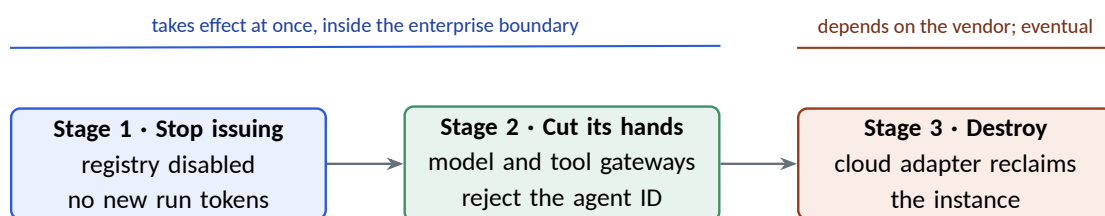


Figure 14 The three-stage kill switch: the first two stages remove the ability to act; the third reclaims resources

The first two stages lie entirely within the enterprise's boundary. Even if the third fails for a while, the agent has already lost the ability to act; what remains is a machine that is running but can do nothing. A normal decommission drains: stop accepting new runs, let compliant runs in progress finish, close pending tickets, then reclaim.

11.5 Interrupt and resume, delivered in three layers

“Interrupt and resume” covers the case where an agent must wait in the middle of a run—for a human approval, for an external callback, for another agent’s result—and the platform saves its state, releases the machine, and continues from the archive when the result arrives. The need is not specific to one kind: a code agent asking a person and a workflow agent waiting for approval both use it.

It is split into three layers delivered in different phases (Table 19). The reasoning: the first layer costs almost nothing but, if skipped, forces Phase 2 to modify every container in production; the second comes with the SDK anyway and also gives long tasks a way to rerun after failure; the real engineering effort is the third layer, and neither of the Phase 1 kinds depends on it—multi-turn Q&A is one run per turn and needs no mid-run interrupt, and most of the cost of waiting is already covered by the vendor’s busy/idle billing.

Table 19 Interrupt and resume in three layers

Layer	Phase	Content
Event format frozen	1	The interrupt event and the resume field enter the container contract; no later change to the contract
Checkpoints in place	1	The SDK shell configures a checkpoint store by default; long tasks can be rerun from the archive on a best-effort basis
Execution mechanics	2	Save the checkpoint, release the machine, route the ticket, resume, continue from the archive in a new session

12

How Each Kind Fulfills the Contract

All four kinds face the same container contract, but who implements it differs (Table 20): Q&A and workflow agents are implemented by platform code, so compliance is guaranteed by construction; code agents are implemented by tenant code through the SDK, so they need declaration plus certification; autonomous agents are implemented by the platform’s shim on behalf of a third-party process.

Table 20 How each kind fulfills the contract

Kind	Loop owner	How the platform implements it	Key boundary
T0	Platform harness	A shared, stateless harness pool; identity injected per request; tool allow-list decided by platform code	The version is the manifest plus the harness version
T1	Platform workflow executor	A six-node workflow language; static analysis of schema, topology and policy	What is signed off must be a declarative workflow
T2	Tenant process	An HTTP-plus-event-stream protocol shell, framework adapters, gateway and network backstops	Capability certified by tests of recovery, rebuild and egress
T3	Third-party process	An orchestrator or shim attached to a fully isolated environment	The shim is not a security boundary; high-risk paths must be distilled into T1

12.1 T0: the platform harness

A Q&A agent does not get its own process. Deployment writes one manifest into the registry, and execution happens in a shared harness pool. The harness is stateless: identity is injected per request, the system instructions and the tenant's prompt are kept in separate segments, and the tool allow-list is decided by platform code—the tenant's prompt is data, and nothing it says can widen the tool set.

Because the harness is the common code of every Q&A agent, it carries a risk that must be governed: one bad harness upgrade stops the largest class of agents at once. A Q&A agent's version is therefore not a single prompt version but the pair “manifest version + harness version”; harness upgrades are canaried, the registry can pin a tenant to a harness version, and the harness itself passes conformance tests. The shared pool also enforces tenant-level concurrency ceilings and allocates the vendor's busy-time bill to tenants by their share of active time.

12.2 T1: the platform workflow executor (Phase 2)

A workflow agent uses a deliberately limited workflow language with only six node types: tool, model, human, branch, parallel, sub-workflow. Branch conditions use deterministic expressions, and each node's input and output are described with JSON Schema. Not being Turing-complete is intentional: only because the language is limited can a workflow be analyzed before release and signed off by risk control.

Compilation has three stages: schema validation, topology validation, and static policy analysis. The most important policy check is dominance: for every high-risk tool node, every path that can reach it must pass through a human approval node; a path that bypasses approval is rejected at compile time.

The platform expressly forbids “convert code to a workflow automatically and run it”. Developers may write code, non-developers may draw workflows, machines may generate drafts, but what is finally signed off must be a declarative workflow that can be verified statically and held to account. Handing risk control a possibly incomplete diagram reverse-engineered from code amounts to forged compliance.

12.3 T2: the tenant container through the SDK

A code agent is implemented in three layers (Figure 15):

- 1. The protocol shell.** A framework-independent HTTP and event-stream service that implements the three endpoints. It turns the tenant's program from a script that exits when finished into a resident service; the port is not exposed publicly.
- 2. The semantic adapter.** It translates a specific framework's execution into the six events and connects the framework's state-saving mechanism to the platform's checkpoint store. The platform maintains adapters for a small number of mainstream frameworks; capabilities a framework lacks are not faked but declared honestly as downgraded.
- 3. The untrusting backstop.** This layer is framework-independent: authoritative tool-call events come from the tool gateway, side-effect scope is enforced by network policy, and capability is certified by test. The first two layers are conveniences for the tenant; the third is the platform's guarantee.

Migrating an existing agent comes down to “three redirections and one entry point”: point the model at the model gateway, point tool calls at the tool gateway, connect state saving to the framework's checkpoint socket, and let the SDK shell provide the entry point. Business logic does not change. The following pseudo-code illustrates the migration boundary only; it does not represent real SDK names:

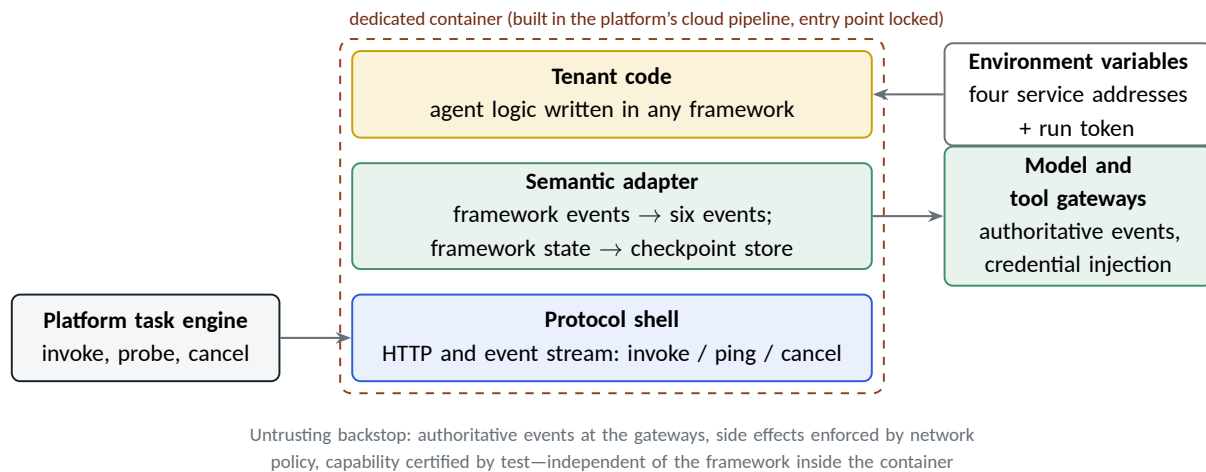


Figure 15 A code agent's three layers: tenant code, semantic adapter, protocol shell—plus a framework-independent untrusting backstop

```
model.base_url <- MODEL_GATEWAY_URL # model via gateway
tools.endpoint <- TOOL_HUB_URL # tools via gateway
checkpoint.backend <- CHECKPOINT_STORE # state to checkpoint store
runtime.agent <- my_graph # business logic unchanged
runtime.serve() # SDK shell: invoke / ping / cancel + events
```

Images are built by the platform in its cloud pipeline; the container's entry point is locked by the build, and the tenant can declare only dependencies and a code directory, never replace the protocol shell; the shell's version is reported with the terminal event. This is not a security boundary (the security boundary is at the gateways and the network) but a guarantee against drift: a certified shell cannot be quietly swapped out at deployment time.

Conformance certification includes at least three destructive tests: whether the process recovers as declared after being killed; whether it rebuilds after its session is reclaimed; and whether an egress block really leaves no bypass. The test suite is not part of the runtime architecture, but it is an indispensable deliverable for admission to production.

12.4 T3: the orchestrator and the shim (Phase 3)

Autonomous agents require separating three things that are often conflated:

- **Topology.** A platform orchestrator with a thin daemon inside the sandbox, or a third-party command-line agent with a platform shim. This is what the kind's definition covers.
- **Product.** An optional cloud development environment, long-lived conversations and multi-person collaboration. This is a product line, not part of the kind.
- **Governance pipeline.** Explore, distil a draft workflow, compile, obtain human sign-off, and finally promote to T1. This is the channel through which an autonomous agent's output enters production safely.

The shim is a thin layer wedged between the platform and the third-party process. It does six things: manage the process lifecycle, translate the third party's output into the six events, forward its permission requests to the platform for decision, carry out platform instructions, hold only short-lived tokens, and handle failure semantics. It explicitly does not make decisions, does not enforce network blocking, and does not interpret

the third party's business semantics. Because it runs on the same machine as an untrusted process, the design must assume it can be compromised; it is not a security boundary. The boundary is full-machine isolation, the model gateway and the controlled network exit. Whole-machine snapshots must be encrypted with a tenant key, carry an expiry, and be scanned for known secret patterns before restore; short-lived tokens are never included in a snapshot and are re-injected on restore.

PART III

Security, Governance and Runtime Control

Constraining untrusted intelligent behavior inside a verifiable boundary: principal identity, the single exits for side effects, the audit ledger, the authoritative store, and the boundary with cloud vendors.

13

Threat Model and Trust Boundary

First, who the platform trusts and who it does not. Tenant code, model output, external files and third-party tool responses are all assumed to be potentially malicious or tainted. The enterprise identity provider, the policy service, the ledger's write path and the gateways' service identities are the controlled roots of trust. The cloud vendor is trusted to provide the compute and storage agreed in the contract, but not to hold the only copy of governance facts, and not to hold final decision rights.

Under that premise, Table 21 lists the main threats, the mitigation for each, and what still needs attention after mitigation.

The "confused deputy" is this: an agent legitimately holds read permission on tool A and outbound-send permission on tool B; induced by an injected instruction, it sends what it read through A out through B. Every step is within its permissions; the combination is a leak. The industry has not fully solved this. The paper reduces it through risk tiers and dual sign-off on dangerous combinations, but does not claim to eliminate it.

The paper does not claim to eliminate covert channels such as DNS, timing or power, nor does it cover the model-weight supply chain, chip-level side channels, or compromise of the enterprise identity provider itself. Those risks fall to infrastructure security, vendor management and the organization's zero-trust program.

14

Identity and Authorization

The agent is a first-class principal. Everything in this section follows from that decision: who the principals are, how permissions intersect, how tokens are separated, and how credentials are kept out of reach.

14.1 Four kinds of principal

The platform recognises four kinds of principal (Table 22). The first three are "who is making the call"; the fourth is "the unit being called". Treating the agent as an independent principal is the starting point of the whole identity design: it borrows neither a developer's account nor a machine's, and it has its own ID, owner and lifecycle.

Table 21 Threat model

Threat	Main mitigation	Still requires attention
Malicious or compromised tenant code	Dedicated containers or micro-VMs, controlled egress, short-lived run tokens only	Kernel and hypervisor escapes
Prompt injection and the cross-tool “confused deputy”	Tool allow-lists, permission intersection, parameter policies, separation of sensitive reads from outbound sends	Legitimate permission combinations being induced into misuse
Credential theft	Credentials fetched only by the tool gateway, briefly; never in the container, the model context or snapshots	Compromise of the gateway’s service identity
Memory and file poisoning	Source tagging, sanitization before writing, expiry, separation of read and write rights	Implicit instructions in unstructured content
Bypassed approval or repeated side effects	Dominance checks, structured interrupt tickets, the tool idempotency ledger	External systems that are not themselves idempotent
Vendor failure or lock-in	Enterprise-held state, the five-primitive adapter, rehearsed exit, the three-stage kill switch	Hosted compute can see runtime plaintext
Audit repudiation or tampering	Platform-assigned sequence numbers, append-only events, hash chain, independent export	Compromise of a root of trust or signing key
Sandbox escape and resource abuse	Micro-VMs, hard resource limits, image scanning, destruction on expiry	Zero-day vulnerabilities and covert channels

Table 22 Four kinds of principal

Principal	Origin	Mode of action	Upper bound on permission
User	An end user, delegated through a business system	On behalf of (OBO)	The user’s own permissions
Service	A business system or back-end service	As a service	The service account’s permissions
Scheduler	A scheduled or event trigger	As itself	No user bound; the task’s creator is recorded
Agent	The hosted execution unit	Determined by the caller’s mode	Constrained jointly by the manifest, grants and structural denials

The scheduler is the principal that is easiest to overlook. When a scheduled task fires there is no “current user”—so what bounds its permissions? The rule here: a scheduled task runs as the agent itself, its upper bound is the intersection of the manifest and the grants, and the task’s creator is recorded for accountability; before each firing the platform checks that the creator is still employed and the agent is not disabled; when the creator leaves, their scheduled tasks are frozen together with the agent and sent for review. The rule prevents “automation nobody owns” from running indefinitely.

Every agent must have a human owner; an owner leaving triggers a review of their agents. The registry entry is the agent’s identity blueprint, and the agent itself holds no long-lived credential.

14.2 The permission-intersection rule

Whether one tool call may proceed is decided by the following intersection:

$$P_{\text{effective}} = P_{\text{manifest}} \cap P_{\text{grant}} \cap \left\{ \begin{array}{ll} P_{\text{user}}, & \text{on behalf of a user} \\ P_{\text{service}}, & \text{as a service} \\ \Omega, & \text{as itself (scheduler)} \end{array} \right. \setminus P_{\text{deny}}$$

Read it as: what the agent declared it could use, intersected with what an administrator granted it, intersected with what the initiator is entitled to do (a scheduler has no “own” permissions, so that term is the universal set Ω and narrows nothing), minus the structural denials. Three intersections mean that any party can only narrow permissions, never widen them: an agent acting for a user never exceeds that user’s own permissions, and an agent granted a tool cannot use it if its manifest does not declare it.

Structural denials are a list of operations that are never available to agent principals (modifying one’s own permissions, cross-tenant access, and the like). They do not live in policy configuration; at the identity layer there is simply no path by which they can be granted.

Every event carries the complete identity chain: tenant, agent, version, session, mode of action, on whose behalf, and approver. Following that chain, an auditor can answer “who did this, on whose behalf, and with whose approval”.

14.3 Two classes of token, each for its own interface

The platform issues two classes of token, each usable only against one class of interface (Table 23, Figure 16).

Table 23 The two token classes

Token	Interface and holder	Restrictions
Platform token	The unified API; held by people, CI and business services	Cannot reach the model gateway, the tool gateway, the checkpoint store or any other runtime exit
Run token	The model gateway, the tool gateway, checkpoints, artifacts; held by a running container	Bound to one run, valid for minutes, restricted to the runtime network; cannot reach the unified API

The design separates “the right to manage the platform” from “the right to act on behalf of one run” structurally. Code inside a container that holds its own run token cannot use it to call the management API and change authorizations; an administrator’s token cannot impersonate a run and call a tool. Leaking either class of token does not directly yield the capabilities of the other.

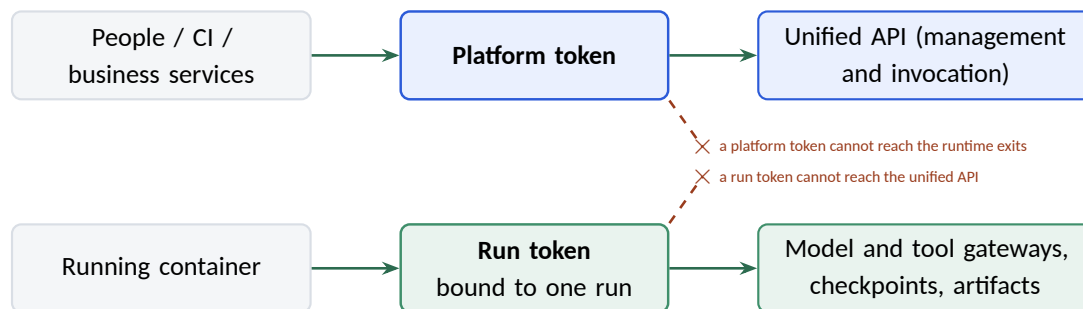


Figure 16 Two token classes, each on its own path: crossing over is always rejected

14.4 Delegation must be verifiable

When a business system calls on behalf of a user, it tells the platform “I represent user U”. If that is merely a string in the request, any business system can impersonate any user, and the permission intersection and audit attribution above become meaningless. The proposal therefore requires that “on whose behalf” be a token issued by the enterprise identity provider and verified by the platform, or the product of a token exchange performed by the platform itself. A bare user ID in a request header never enters the identity chain.

14.5 Tool risk tiers

The tool gateway keeps a four-tier risk dictionary: read internal data, write internal data, send externally, and high-risk write. The tiers serve two immediate purposes: Q&A agents may not receive outbound-send tools by default; and granting an agent both “read sensitive data” and “send externally” requires two signatures and is flagged in audit. This is the mitigation for the confused-deputy risk described above.

14.6 The credential vault

The secrets that tools need (API keys, account passwords) are kept in the credential vault, which follows three rules:

1. People may write, rotate and revoke credentials, but never read them in plaintext.
2. The plaintext read path is open only to the tool gateway’s service identity, and on the network the vault is reachable only from the tool gateway.
3. Secrets never enter the model’s context and never appear in tool results, logs or snapshots.

Figure 17 shows how a credential moves during one tool call: the agent declares a tool call, the gateway checks authorization, fetches the credential briefly from the vault, calls the target system on the agent’s behalf, and returns the result. From beginning to end the agent sees only the tool’s name and its result.

15

Shared Services: The Governed Exits

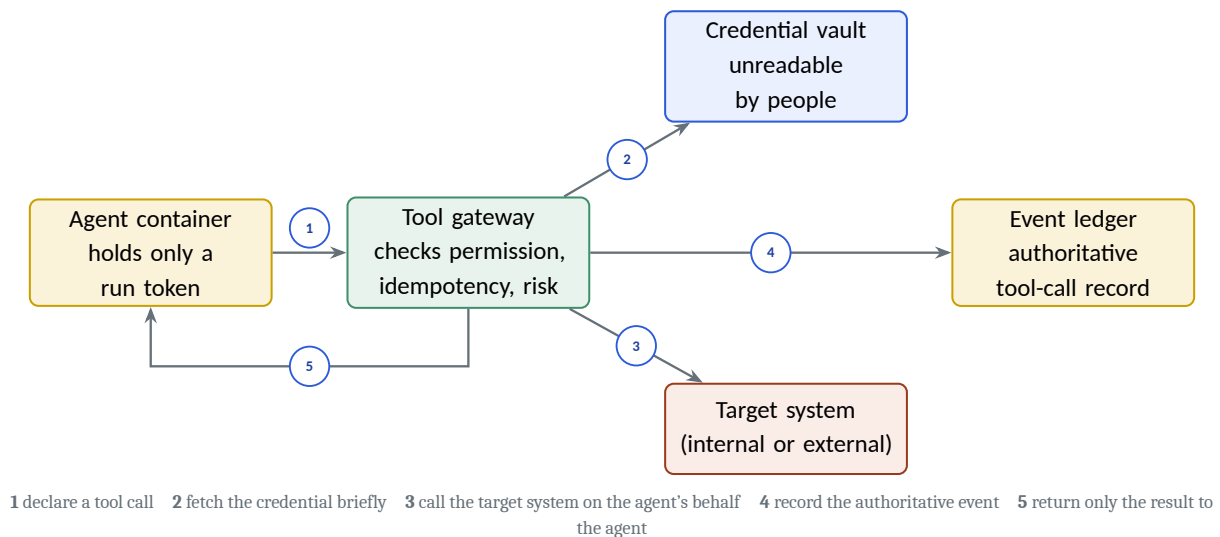


Figure 17 Credential flow during one tool call: the credential appears briefly at the tool gateway and never enters the container

15.1 Why single exits

Everything an agent does to the outside world happens in three kinds of action: calling a model, calling a tool, executing code. If those actions could leave the container directly, the platform would neither know they happened nor be able to stop them. The shared-services layer gives each of the three exactly one path, with the platform's checks, records and refusals on that path. This layer runs in the opposite direction to the entry layer: the entry layer is the outside calling the platform; shared services are running agents calling enterprise capabilities.

Figure 18 shows the shared-services region of the complete architecture: four modules, two of which reuse the enterprise's existing gateways and two of which are built-in-house.

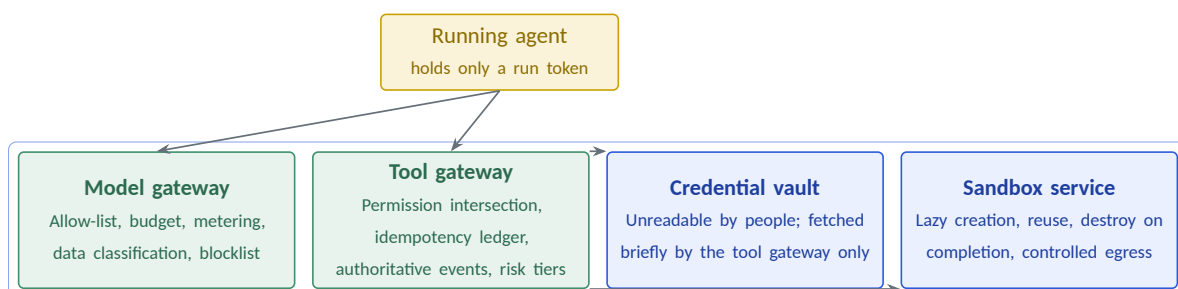


Figure 18 The four shared-services modules: the agent touches only the two gateways; the credential vault and the sandbox service are called by the tool gateway on its behalf

15.2 The model gateway

The model gateway is the only exit for every large-model call. It is responsible for the model allow-list, the usage budget (exhaustion ends the run in the “budget exceeded” state), per-call metering, routing among models, the mapping from data classification to permitted models (tenants handling sensitive data may use only designated model domains), and the agent blocklist (the second stage of the kill switch). Phase 1 provides at least one global default rule for data classification; finer-grained data-loss prevention comes in Phase 2. A cloud vendor's own model gateway does not enter the authoritative chain, because it holds the decision over which model may be used and it hosts the model keys.

15.3 The tool gateway

The tool gateway is the only exit for every tool call and the source of the authoritative tool-call events. “Tool gateway” is the unified name in this document; some interfaces retain the earlier name Tool Hub (for example the environment variable `TOOL_HUB_URL`) as a compatibility alias.

Tools fall into four classes, of which only the last needs a sandbox:

1. **Skill descriptions.** Capability descriptions loaded as text into the agent’s context; no external call is involved.
2. **API tools.** Direct calls to a target business service.
3. **MCP tools.** Provided by an independent tool-server container through the Model Context Protocol.
4. **Environment-interaction tools.** Executing code, running commands, driving a browser or a desktop. These need an isolated, ephemeral environment.

The call path for environment-interaction tools is shown in Figure 19: the agent calls the tool gateway, the gateway calls the platform’s sandbox service, and the sandbox service asks the vendor for an isolated environment through the cloud adapter. Even between the platform’s own components, the call goes through the platform’s abstraction rather than straight to the vendor.

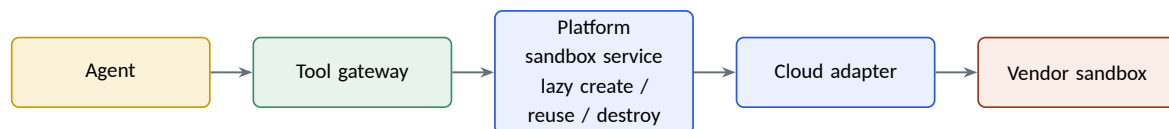


Figure 19 Call path for environment-interaction tools: platform components call each other through the platform’s own abstraction, never straight to the vendor

Phase 1 adds to the enterprise’s existing tool gateway: routing by the four tool classes, injecting the gateway address into containers, the idempotency ledger, the agent blocklist, and a minimal risk dictionary.

15.4 The sandbox service

The sandbox service is a lightweight component built in-house that manages the lifecycle of isolated environments: create only when needed, reuse within a session, destroy on completion, and converge egress onto a controlled exit. It is exposed through its own `/v1/sandboxes` interface; the tool gateway is only one of its consumers, and tenants can request a sandbox directly to execute code. Credentials never enter a sandbox; when code in a sandbox needs a protected resource, the call still goes through the tool gateway.

15.5 A statement of the trust boundary

The SDK inside a container, the autonomous-agent shim and the events an agent reports about itself are not roots of trust. The security boundary is formed jointly by the enterprise’s gateways, the controlled network exit, short-lived identity, sandbox isolation and the enterprise-held ledger.

16

The Unified Ledger, Metering and Observability

16.1 What the ledger is

The unified ledger is an append-only record of events held by the enterprise. Every run, every tool call, every policy decision and every administrative action is written as an event with a sequence number and an identity chain. It serves three consumers at once: the user interface reads event streams from it, audit replays chains of responsibility from it, and metering computes cost from it. Because all three consume the same data, they never disagree.

16.2 Each class of fact has its own authoritative source

The same event stream can be consumed by many parties, but each class of usage data must respect its own authoritative source (Table 24). The principle underneath: data reported by a container about itself is supplementary only, never the basis for billing or audit.

Table 24 Authoritative sources of each class of fact

Fact	Authoritative source	Reason
Model usage	The model gateway	It holds the real request and response counts
Tool calls and side effects	The tool gateway	It sits at the exit for authorization, idempotency and external calls
Runtime busy time	The vendor's bill	It determines what the enterprise actually pays
Sandbox duration	The platform sandbox service plus the vendor	The platform holds the request semantics, the vendor holds the resource bill
Container self-reports	Supplementary only	A container is not a trusted source for metering or audit

16.3 Three books of metering

Metering keeps three books (Figure 20): the internal allocation book answers which tenant, agent and run should bear what cost; the vendor book is what the enterprise actually pays to model, cloud and tool vendors; the reconciliation book explains the differences between the two, unallocated costs and estimation error. All three are computed from the same event stream.

Two implementation requirements follow. First, the circuit breaker's spend ceiling and the metering estimate must use the same counter; otherwise the budget system may believe the ceiling has not been reached while the task engine keeps running. Second, reconciliation granularity depends on the vendor: if the vendor's bill supports tags per session or instance, reconciliation is per run; if not, it can only be per tenant, and the documentation must say so rather than overstate.

The cost of the shared Q&A pool is allocated to tenants by their share of active time:

$$C_{\text{tenant, pool}} = C_{\text{pool, busy}} \times \frac{T_{\text{tenant, active}}}{\sum_i T_{i, \text{active}}}$$

The full cost of one run has seven components:

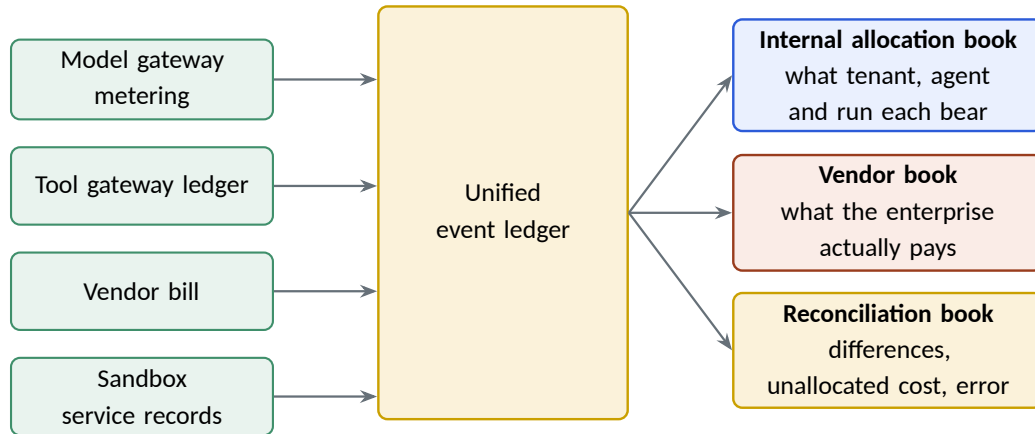


Figure 20 Three books of metering, all computed from the same event stream and reconcilable against each other

$$C_{\text{run}} = C_{\text{model}} + C_{\text{tool}} + C_{\text{runtime}} + C_{\text{sandbox}} + C_{\text{storage}} + C_{\text{network}} + C_{\text{platform}}$$

16.4 Operating metrics

As a reference architecture, the paper sets no success-rate, latency or availability targets that have not been measured. After implementation, at least six metrics should be tracked together: the number of production runs completed successfully, cost per run, policy denial rate, audit event completeness, human takeover rate, and the difference against vendor bills. Formal service-level targets should be set by business tier only after capacity tests, failure drills and verification of vendor capabilities. “How many agents are onboarded” or “how many calls were made” on their own hide failures, privilege violations and waste.

16.5 Three layers of observability

Observability has three layers, built in three different ways:

- 1. Instrumentation.** Platform code uses the OpenTelemetry open standard throughout; every tool call is a span, and the trace identifier propagates end to end. Instrumentation lives in the enterprise’s own code and is done once.
- 2. Collection.** A container’s standard output and error streams go first to the platform’s collector, are redacted, and only then are forwarded to the observability backend; the vendor’s native log retention is set to the minimum or switched off. This keeps prompts and user data from lingering as logs on the vendor’s side.
- 3. Dashboards.** In Phase 1 the cloud vendor’s observability backend may be rented, because the instrumentation follows an open standard and switching means changing a destination address; tenants, however, view their own traces only through the platform, never directly in the vendor’s dashboard.

Trace attributes default to an allow-list: prompts, model output and sensitive tool parameters do not enter dashboards. Monitoring answers “is the machine healthy”; the ledger answers “who did what”. Accountability always rests on the enterprise ledger.

16.6 Tamper evidence and compliant deletion

The ledger is append-only. Each event includes the hash of the previous event, forming a hash chain; the chain's daily digest is anchored to versioned-locked or write-once storage. Altering any historical event breaks the chain check, which is the technical basis on which auditors can rely on the ledger.

Compliant deletion (a user exercising a right to erasure, for example) looks incompatible with an append-only ledger. The resolution is per-thread encryption: each thread's event bodies are encrypted under their own key; deletion destroys the key, making the bodies unreadable while the sequence, types and audit structure of the events remain intact.

17 The Authoritative Store

17.1 One logical database, semantics first

Phase 1 uses one logical PostgreSQL database for the event ledger, checkpoints, the registry, and the mapping between threads and sessions. The point is to freeze one consistent data semantics first, not to pack every workload into one database forever. When events are later moved to a message queue, analytics to a columnar store and checkpoints to dedicated storage, no contract above them needs to change.

Figure 21 shows the four classes of data this logical database holds. What they have in common is authority: any module that needs to answer “what really happened” consults this store.

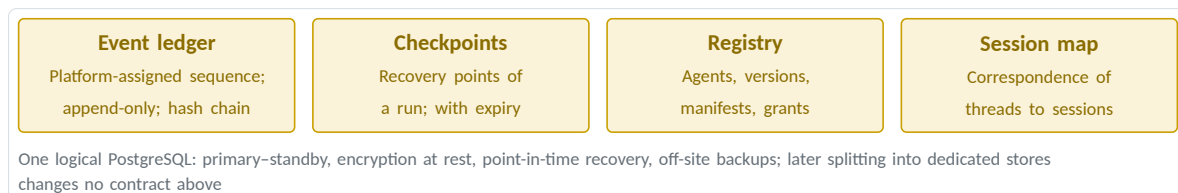


Figure 21 The four classes of data in the authoritative store (the storage region of the complete architecture)

17.2 Capacity and write amplification

Output fragments (`message.delta`) are the main source of write amplification. A rough estimate: at a peak of 2,000 conversations per hour with about 100 fragments each, that is 200,000 rows per hour and close to 150 million rows in 30 days; add checkpoints of several hundred kilobytes per step for long tasks, and a single database is exhausted by capacity before performance. Phase 1 should therefore adopt from the start:

- Output fragments go to a short-retention stream table, kept for 7 days by default.
- When a run completes, its fragments are merged into long-retention message events.
- Checkpoints are retained for 7 days by default, shortened to 24 hours once the run completes.
- Large objects are referenced in object storage, never inlined in the database.
- The database runs primary-standby with encryption at rest, point-in-time recovery and daily off-site backups.

Recovery objectives must be set after capacity tests, backup restores, regional failure drills and vendor-outage rehearsals. The paper pre-fills no unverified recovery-point or recovery-time targets.

18

The Execution Plane and the Vendor Boundary

This section corresponds to the bottom two layers of the complete architecture (Figure 22): in the execution plane the shells and the adapter are the platform’s own and the machines on the two product lines are rented; the cloud resources below are rented entirely by usage. A single dividing line explains both layers—vendor names appear only at the very bottom, and nothing in the execution plane depends directly on any one vendor.

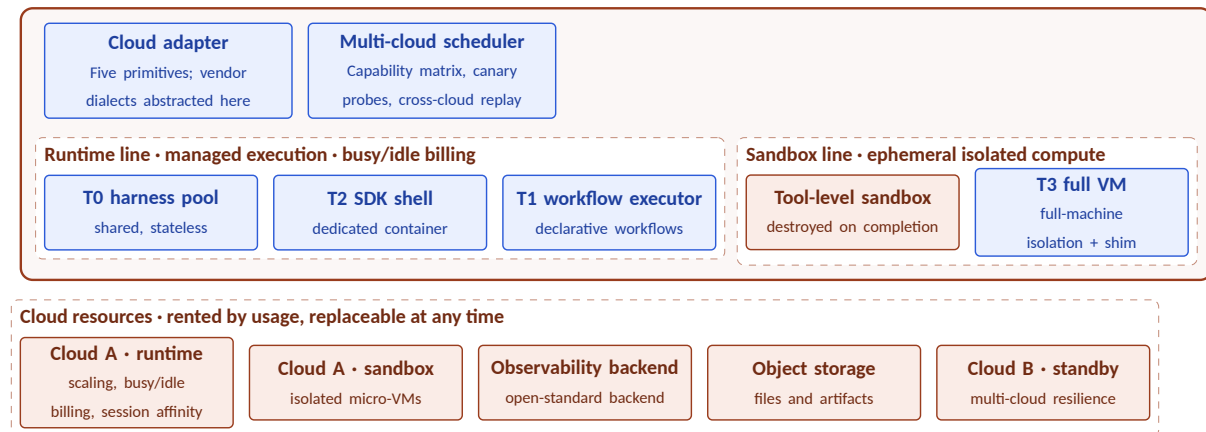


Figure 22 The execution plane and the cloud resources (the bottom two layers of the complete architecture)

18.1 The cloud adapter: five primitives

The platform uses cloud resources through exactly five primitives: deploy, invoke, cancel, status, destroy. No other part of the platform depends directly on a specific vendor. The shape of those five primitives is the reservation for multi-cloud: switching vendors means implementing one new adapter.

Three capabilities that the vendor’s runtime provides must nonetheless be abstracted inside the adapter: session affinity, traffic switching and busy/idle reporting. They are each vendor’s “dialect”—their semantics differ from vendor to vendor—and letting them reach the task engine would create hidden rework when the vendor changes.

18.2 Liveness probes and the authoritative busy/idle judgment

The container’s /ping means only “still alive”. The authoritative busy/idle judgment comes from the task engine: does this session have an unfinished run? The cloud adapter reports busy or idle to the vendor on that basis. The design has two purposes: to stop a container from reporting “busy” forever so that the vendor never reclaims it (idle billing), or reporting “idle” so that the vendor reclaims an instance mid-run; and to guarantee honest idle reporting, which is the precondition for the vendor’s busy/idle billing to actually save money. The maximum run duration in the manifest is enforced by a platform timer: on expiry the platform sends cancel, and after a grace period destroys the instance, without relying on the container’s cooperation.

18.3 The two product lines

The execution plane rents exactly two product lines (Table 25). Vendor names label only the source of the rental; they are not components of the execution plane.

Table 25 The two product lines of the execution plane

Product line	What runs on it	Billing and isolation
Runtime	The Q&A harness pool, the code-agent SDK shell, the workflow executor	Managed long-running execution, billed by busy/idle time
Sandbox	Tool-level code and browser execution, the autonomous agent's full VM	Ephemeral compute, billed per use or per hour, destroyed on completion

18.4 Three verdicts on vendor capabilities

Measured against the three criteria of Section 4 (no decision rights, no authoritative data, a standard exit), a vendor's complete product line sorts into three verdicts (Table 26).

Table 26 Three verdicts on vendor components

Verdict	Components	Reason
Use	Runtime, sandbox, object storage, read-only observability backend	Mechanical resources with no final decision rights; observability follows an open standard and is replaceable
Do not use	Cloud model and tool gateways, credential hosting, identity service, knowledge base, memory service, low-code platforms, hosted guardrails	They hold decision rights, secrets or authoritative data, creating deep lock-in
Candidate	A second vendor's runtime, specialized sandbox products	Enter through the adapter and the capability matrix, without changing the contract above

18.5 Procurement veto items

Vendor verification covers at least six items: instance-level or network-level egress convergence, session affinity, VM snapshots, billing tag granularity, whether native logs can be disabled or shortened, and whether busy/idle semantics can be aligned with the platform's judgment. Any item that affects the ten invariants or the ability to leave should be a procurement veto, not an engineering patch after launch.

18.6 Multi-cloud is not double deployment

The Phase 2 multi-cloud scheduler is responsible for the capability matrix, canary probes, circuit-breaking rerouting and cross-cloud replay. The rule is “one session stays with one vendor for its lifetime”, so that in-memory state is never shared across clouds; resilience relies on enterprise-held checkpoints, artifacts and the ledger—recreate the session on another cloud and replay from the archive—rather than on maintaining an expensive hot-hot pair.

PART IV

Productization, Delivery and Evolution

Turning the architecture into a reusable product: integrating with enterprise systems, separating kernel from policy packs, what Phase 1 delivers, how it is accepted, and what comes next.

19 Enterprise Integration and Generalization

19.1 Reusing what the enterprise already has

The platform is not an island. An enterprise usually already has an identity provider, a model gateway, a tool gateway, a continuous-integration pipeline, finance systems and an event bus; all of them are integrated rather than rebuilt (Table 27). For enterprises that do not yet have a model or tool gateway, the platform ships default implementations (an open-source LLM gateway and a built-in tool gateway). Figure 23 is the enterprise-systems layer of the complete architecture, with each system annotated with the platform module it connects to.

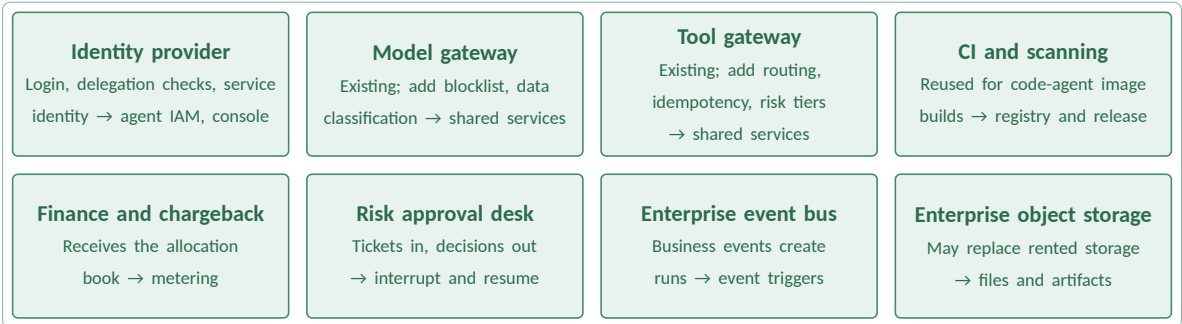


Figure 23 Enterprise systems and the platform modules they connect to (the enterprise-systems region of the complete architecture)

19.2 Kernel, adapter packs and policy packs

For one architecture to be reused across enterprises and industries, what stays fixed and what is replaced per enterprise must be stated up front. The platform is divided into three layers (Table 28).

The kernel contains no industry business logic; that is the key to generalization. A new enterprise goes live in four steps: bind the identity provider and choose a cloud adapter; integrate existing gateways or use the defaults; fill in the kind presets and the risk dictionary; run the ten-invariant acceptance and the conformance tests. The kernel code does not change.

Table 27 How enterprise systems are integrated

Enterprise system	Platform module	How	Phase
Identity provider	Agent IAM, console, CLI	Login for people and CI, delegation verification, service identity	1
Model gateway	Shared services	Add the agent blocklist and data-classification mapping	1
Tool gateway	Shared services	Add four-class routing, the idempotency ledger and the risk dictionary	1
CI and dependency scanning	Registry and release	Reused when building code-agent images	1
Finance and chargeback	Metering and reconciliation	Export the allocation book; automated posting later	1 / 2
Risk approval desk	Interrupt and resume	Push tickets; decisions resume the run	2
Enterprise event bus	Event triggers	Business events create runs	2
Enterprise object storage	Files and artifacts	May replace rented object storage	optional

Table 28 Kernel, adapter packs and policy packs

Layer	Content	Rate of change
Kernel	The three interface layers, the five action channels, the four-axis manifest, the task engine and circuit breaker, the unified ledger and hash chain, the identity core, the five-primitive adapter, the three books, the three-stage kill switch	Fixed across enterprises, strictly versioned
Adapter packs	Identity-provider binding, model and tool gateway integration, cloud adapters, the observability backend	Replaced per enterprise and per vendor
Policy packs	The risk dictionary, approval rules, data classification, structural denials, retention and deletion policy	Vary by industry, region and risk appetite

19.3 An illustrative policy pack

The following example shows how industry constraints enter a policy pack without contaminating the kernel; it corresponds to no real organization. A healthcare provider allows agents to coordinate appointments, pre-screen documents and check insurance details. Its policy pack states that protected health information may enter only designated model domains; that sending a medical summary externally requires human confirmation; that altering a diagnosis, signing a prescription, widening one's own permissions and cross-tenant access are structural denials; and that high-risk workflows may be carried only by workflow agents or specially reviewed code agents, with per-step checkpoints, cold interrupts and a complete audit chain.

Scope of the example: this is a fictional policy pack and constitutes no medical, privacy or compliance advice. A real implementation must have its data classification, approval conditions and denials defined by the legal, security and business owners of the jurisdiction concerned.

20

Phase 1 Delivery, Roadmap and Acceptance

20.1 Three things that must be right on day one

Three things, if wrong, can only be fixed later by rewriting every agent in production, so they must be right from day one:

1. **Freeze the contract.** Define the format of the six events, including the interrupt event and the resume field, once and completely.
2. **Own authoritative state.** Events, checkpoints, the registry and the session map land in the enterprise's own database from the first day.
3. **Both gateways in the path.** From the first day there is no route around the gateways for model or tool calls.

20.2 Scope by phase

The order of evolution is “freeze the contracts that cannot be reworked, then widen governance, then open up more autonomy” (Figure 24, Table 29). The boundaries between phases are not counts of features; they are thresholds at which the risk boundary is opened one step further.

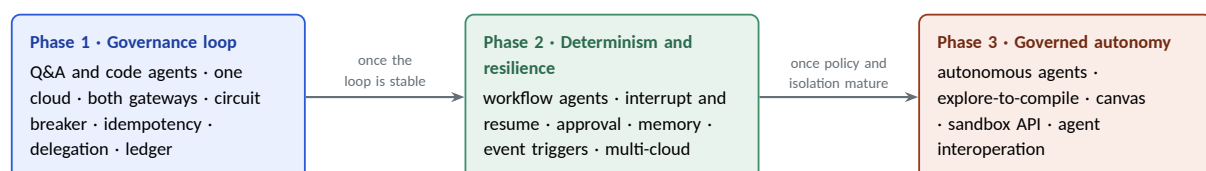


Figure 24 From a governance loop to governed autonomy

20.3 Six acceptance loops

What counts as “built” in Phase 1? Not “all components deployed” but six end-to-end paths that run from the entrance to the final evidence (Table 30). The component list is for building; the loops are for acceptance.

21

Key Architecture Decisions and Trade-offs

Table 31 summarizes the twelve most important decisions in this paper, each with what it gains and what it costs, so that reviewers can challenge them one by one.

One order runs through all of these decisions: build identity and the ledger before memory and smart scheduling; prove least privilege and the exit path before opening more tools and more autonomy.

Table 29 Scope of the three phases

Phase	Core capabilities	Product boundary
Phase 1	Q&A and code agents, one cloud, scheduled triggers, basic console, circuit breaker, both kinds of idempotency, two token classes, delegation verification, three-stage kill switch, hash chain, log intake, fragment stream table	A complete loop from development to accountability
Phase 2	Workflow agents, interrupt mechanics, approval console, quota management, long-term memory, event triggers, multi-cloud scheduling, own observability dashboards, advanced console	Deterministic workflows and operational resilience
Phase 3	Autonomous agents, the explore-to-compile pipeline, a workflow canvas, a sandbox API product, agent interoperation	More autonomy inside the same governance boundary

Table 30 The six acceptance loops of Phase 1

#	Loop	End-to-end acceptance
1	Develop and release	Few changes, cloud build, release; goes live after the conformance check and can be rolled back
2	Conversation	A question or a scheduled trigger, streamed output, resumption by sequence number after a disconnect
3	Tool execution	Gateway authorization, short-lived credential injection, four-class routing, a sandbox opened on demand for environment tools
4	Audit and accountability	Full replay from the ledger, attribution through the identity chain, administrative actions linked, hash chain verified
5	Cost reconciliation	Allocation per agent and per run is explainable and reconciles against the vendor's busy/idle bill
6	Safety	A runaway loop is broken by the circuit breaker; an unauthorized call is refused; a duplicate creation returns the same run; a forged delegation is rejected; the first two kill-switch stages still work with the vendor unreachable

22

Residual Risks and Companion Specifications

22.1 Risks accepted and continuously reduced

Some risks are reduced by the design but not eliminated. Table 32 lists them plainly; reviewers and readers should hear them stated rather than discover them.

Table 31 Key architecture decisions

ID	Decision	Gains	Costs and constraints
A1	Adapt frameworks through a protocol	Frameworks replaceable, contract stable	Adapters and a conformance suite must be maintained
A2	Complete outside, minimal inside	Product experience decoupled from runtime stability	The control plane carries more semantics
A3	Four axes instead of hard-coded kinds	New scenarios by combination; orthogonal policy	Stricter manifest and admission checks
A4	Record first, then distribute	Resumable reads, ordered events, replay	The ledger becomes a critical write path
A5	Gateway events are the authoritative facts	No reliance on untrusted container self-reports	Gateways must be highly available and low-latency
A6	Circuit breaker in Phase 1	A loss ceiling from the first launch	Counters and termination logic to build
A7	Two token classes	Management and runtime structurally separated	More complex token exchange and validation
A8	Three-stage kill switch	Shutdown independent of the cloud vendor	Registry and both gateways must act together
A9	One database first, semantics first	Fast delivery with consistent semantics	Fragment write amplification must be managed early
A10	One cloud in practice, multi-cloud by contract	Small operational surface at the start	Portability must be proven by repeated rehearsal
A11	Autonomous agents deferred to Phase 3	Mainstream, governable workloads first	The most autonomous scenarios are not yet covered
A12	Policy packs isolate industry logic	The kernel is reusable across industries	Each industry maintains its own policy assets

Table 32 Residual risks

Risk	Reduction	Why it remains
Cross-tool confused deputy	Risk tiers, dual sign-off on sensitive-read plus outbound-send, parameter policies	A legitimate combination of permissions can still be exploited by injected instructions
Implicit instructions in files	Content isolation, scanning, least privilege for tools	Unstructured files can carry malicious instructions
Data leakage through external models	Data classification mapped to model allow-lists, log redaction	Phase 1 lacks fine-grained data-loss prevention
Vendor reconciliation granularity	Tag verification, the three internal books	Granularity is ultimately limited by the vendor's billing
Vendor visibility of runtime plaintext	Egress convergence, minimal data, contractual and regional constraints	Hosted compute inherently processes plaintext

22.2 Companion specifications

This paper describes the architecture. Field-level definitions are carried by three companion specifications, maintained separately with their own version numbers: the field-level specification of the container contract (sequence numbers, event de-duplication identifiers, liveness semantics, format versions, error classes), published as OpenAPI and JSON Schema; the detailed design of the tool authorization model and identity; and the conformance test suite, with at least three cases—process kill, session reclaim and rebuild, and egress blocking. Of the three, the container-contract specification has the highest priority because it cannot be reworked; until it is frozen, onboarding several agent frameworks in parallel should not begin.

23

Conclusion

The engineering scope of an enterprise agent platform is far larger than “put the agent in a container”. The model decides how an agent reasons; the runtime decides where it runs; the control plane decides under whose identity, within what boundary, and at whose cost and accountability it acts.

The architecture proposed here replaces framework binding with a protocol, replaces hard-coded kinds with a four-axis capability manifest, separates resources, conversations and executions through sessions, threads and runs, closes side effects behind five action channels, composes identity security from two token classes, verified delegation, a credential vault and a three-stage kill switch, connects events, audit, metering and recovery through a unified ledger, and keeps the freedom to choose vendors through a five-primitive cloud adapter.

What matters in this design is not the number of components but whether the boundary can be proven: that no long-lived secret ever entered a container, that no side effect bypassed the exits, that events were recorded before delivery, that permissions could only narrow, that cost had a hard ceiling, and that the agent could still be cut off with the vendor unreachable. Only when those ten invariants hold does the platform truly possess control.

For an enterprise this means three concrete things. Business teams can put agents to work faster, because onboarding, compute and permissions no longer have to be solved by each team. Security and audit can approve production use with confidence, because every action has an exit, a record and a ceiling. And management does not have to trade control for speed, because everything beneath the contract can be replaced.

An enterprise may rent all the compute and environments its agents need. Identity, keys, decision rights, facts and the ability to leave should remain its own.

Appendix A

Glossary

Terms are listed in roughly the order they first appear; first-time readers may find it useful to read the whole list once.

Term	Meaning
Agent	A versionable software unit that can be given an identity and that, within one run, chooses models or tools from its input and produces a result. Not a model and not a prompt.
Controllable / meterable / observable / auditable	The platform's four target properties: it can limit what an agent does, account for what each run cost, see whether the system is healthy, and replay who did what and when.
Control plane / execution plane	The decision-and-record part the enterprise owns (identity, authorization, tasks, the ledger) versus the replaceable compute part where agents actually run.
Harness	The general execution loop implemented by the platform, which calls models and orchestrates tools. All Q&A agents run on it.
T0 / T1 / T2 / T3	The four kinds of agent, divided by who decides the next step: Q&A ("answer from the script", the platform's fixed loop), workflow ("follow the form", a workflow drawn in advance), code ("bring your own program", the tenant's own code), autonomous ("let the expert work", a ready-made autonomous agent).
Capability manifest	The declaration submitted when an agent is registered: values on the four axes, a streaming flag, a maximum run duration and the harness version.
Four axes	The four independent dimensions of the manifest: loop owner, isolation, side-effect scope, recovery.
Conformance certification	The mechanism by which a declared capability takes effect only after destructive tests (recovery after a process kill, rebuild after session reclaim, egress blocking).
L1 / L2 / L3	The three interface layers: the public unified API; the minimal contract between platform and container; the service exits through which containers reach enterprise capabilities.
Run	One execution with a start, a terminal state, a budget and the ability to be canceled; the unit of metering and audit.
Thread	A long-lived conversation history that may span many sessions and runs.
Session	A short-lived lease on an isolated execution environment; occupies a machine while active and may be released when idle.
Checkpoint	A safe point saved during a run, from which execution can continue after a failure instead of restarting.
Interrupt and resume	When an agent must wait mid-run (for approval, for a callback), the platform saves its state, releases the machine, and continues from the archive when the result arrives.
Action channel	One of the five paths through which a running agent can affect the outside world: being called, calling a model, calling a tool, asking a person, calling another agent.
Idempotency	Repeating an operation produces no additional effect. The platform has creation idempotency (a retry does not create several runs) and side-effect idempotency (a retry does not perform a real action twice).
Circuit breaker	Hard ceilings on a run's steps, calls, model usage, duration and spend; exceeding any one terminates the run.
Effectively once	The guarantee composed from a checkpoint before each effect, the tool gateway's idempotency ledger and archived results: a real-world action is not repeated by a retry.

Term	Meaning
Three-stage kill switch	Stop issuing tokens, have the gateways reject the agent, destroy the instance; the first two stages do not depend on the cloud vendor.
On-behalf-of (OBO) delegation	A business system calling on behalf of an end user; “on whose behalf” must be a token issued by the identity provider.
Identity provider (IdP)	The enterprise’s central login and identity-issuing system, such as a single sign-on service.
Platform token / run token	The two token classes: the first for people, CI and business services against the unified API; the second for running containers against the model and tool exits, bound to one run and valid for minutes.
Permission-intersection rule	Effective permission is the intersection of the manifest, the tool grants and the initiator’s own permissions, minus structural denials; any party can only narrow, never widen.
Structural denials	Operations never available to agent principals; at the identity layer there is no path by which they can be granted.
Credential vault	The central store for tool secrets: people may write, rotate and revoke but not read; only the tool gateway may fetch, briefly.
Model gateway / tool gateway	The single exits for all model calls and all tool calls respectively, and the places where the platform checks, records and refuses.
Sandbox	An isolated execution environment destroyed after use, for environment-interaction tools such as running code or driving a browser.
Unified ledger	The enterprise-held, append-only record of events that serves the user interface, audit and metering together.
Hash chain	Each event includes the hash of the previous one; altering any historical event breaks verification.
Three books	The internal allocation book, the vendor book and the reconciliation book.
Cloud adapter	The anti-corruption layer that translates the platform’s five primitives (deploy, invoke, cancel, status, destroy) into vendor APIs.
Kernel / adapter packs / policy packs	What stays fixed across enterprises; what is replaced per enterprise and vendor; what varies by industry and risk appetite.
OpenTelemetry	The open standard for observability data—instrumentation, data format and transport; adopting it makes switching monitoring backends a change of destination address.
MCP	The Model Context Protocol, an open standard for connecting tools to large-model applications.

Appendix B

L2 Container Contract Quick Reference

Class	Item	Semantics
Endpoint	POST/invoke	Start or resume a run; returns a server-sent event stream
Endpoint	GET/ping	Liveness probe only; busy/idle is judged by the task engine
Endpoint	POST/cancel	Request exit at a safe point; the cloud adapter destroys after the grace period

Class	Item	Semantics
Event	message.delta	Streamed output fragment; short retention, then merged into a message
Event	tool.call	Tool call declared by the container; the tool gateway's record is authoritative
Event	checkpoint	A recovery point of the run, with its capability level
Event	interrupt	A request-for-input ticket; format frozen in Phase 1, mechanics in Phase 2
Event	artifact	A reference to a large object; content is never inlined
Event	run.status	State, error class, harness version
Environment variable	MODEL_GATEWAY_URL	The single exit for models
Environment variable	TOOL_HUB_URL	The single exit for tools
Environment variable	CHECKPOINT_STORE	Address of the checkpoint service
Environment variable	ARTIFACT_STORE	Address of the files and artifacts service
Manifest	Four axes, streaming, maximum duration, harness version	Declares capability, limits and the harness version; effective after conformance certification
Error class	retryable / terminal / policy_denied / budget_exceeded	Retryable, not retryable, denied by policy, budget exceeded

Appendix C

Ten-Invariant Acceptance Checklist

#	Acceptance item	Status
1	No long-lived secret exists in containers, images, logs or snapshots	☐
2	Direct calls from a container to a model, a tool or the internet all fail; side effects cannot bypass the governed exits	☐
3	Tool-call audit relies on tool-gateway events and can be compared with container self-reports	☐
4	The enterprise-side ledger is append-only, its hash chain verifies, and tampering with a historical event is detected	☐
5	Every run has enforceable hard ceilings on steps, calls, model usage, duration and spend	☐
6	A platform token and a run token used across interfaces are both rejected	☐
7	A forged delegation fails verification against the identity provider	☐
8	With the vendor's API simulated as unavailable, the first two kill-switch stages still take effect	☐
9	Sequence numbers are assigned by the platform and a client resumes from them after a disconnect	☐

#	Acceptance item	Status
10	Administrative actions and agent actions can be replayed in one identity chain	☐

Appendix D

Revision History

Date	Version	Notes
2026-09-17	Design baseline v3.0	Internal final design document and overall architecture finalized
2026-09-18	Technical white paper v1.0	Reference architecture for publication: related work, one request end to end, threat model, complete architecture with per-section detail views, glossary; unified terminology; unmeasured figures removed

References

- [1] Amazon Web Services. *Amazon Bedrock AgentCore FAQs*.
<https://aws.amazon.com/bedrock/agentcore/faqs/> (accessed September 2026)
- [2] Anthropic. *Claude Platform Documentation: Managed Agents*.
<https://platform.claude.com/docs/en/home> (accessed September 2026)
- [3] OpenAI. *Agents SDK and Agents Platform Documentation*.
<https://developers.openai.com/api/docs/guides/agents-sdk> (accessed September 2026)
- [4] Google Cloud. *Gemini Enterprise Agent Platform: Agent Runtime*.
<https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/runtime> (accessed September 2026)
- [5] Microsoft. *Configure agent identity security with the Zero Trust Assessment*.
<https://learn.microsoft.com/en-us/security/zero-trust/assessment/ai-pillar> (accessed September 2026)
- [6] LangChain. *LangSmith Deployment*. <https://docs.langchain.com/langsmith/deployment> (accessed September 2026)
- [7] Temporal. *Durable AI*. <https://docs.temporal.io/ai> (accessed September 2026)
- [8] Tencent Cloud. *CubeSandbox: Secure MicroVM Sandbox for AI Agents*.
<https://github.com/TencentCloud/CubeSandbox> (accessed September 2026)